Verified Stopping, Not Adaptive Allocation, Saves Reads in Multi-Container DNA Storage: A Leak-Free Computational Study on a Digital Twin and Real Read Streams

Sai Mouli
saimouli12345678@gmail.com

***Abstract*—When an erasure-coded DNA archive is spread over several separately synthesised and handled containers of unknown quality, two retrieval decisions arise: when to stop sequencing, and which container to read next. We find that the first is worth an order of magnitude more than the second. All results are computational: no DNA was synthesised or sequenced for this paper, and the real-read experiments replay public Illumina runs. On the DT4DDS digital twin with six independently synthesised, aged and amplified containers, a streaming decoder that stops only when every stripe, its CRC-32 and the file's SHA-256 verify consumed 3.49k reads per retrieval (95% CI 3.37–3.60k), against the 4.80–5.15k that the same read proportions must *reserve* in advance to succeed on 99% of retrievals: stopping turns a reservation into a bill 27–32% smaller, with no model of the containers. Choosing *which* container to read next is a far smaller and more fragile effect. Under our pre-registered container model, a single efficiency multiplier, every model-based rule — Thompson sampling, UCB, learn-then-commit, information-greedy — read *more* than round-robin when one container was near-dead (+7% to +441%), because one multiplier cannot express dropout; a hindsight-tuned static allocation with the same stopping rule was 4.2% cheaper than round-robin. Adding a dropout parameter to the container model removes most of that penalty (UCB −3.5%, 95% CI −4.9 to −2.2%), which locates the failure in the model rather than in adaptivity, but the best allocation gain remains 3.5% against stopping's 27–32%. No rule lowered the 0.99-quantile in any configuration, including with the better model, so none of this reduces reserved capacity. Over 23,160 end-to-end retrievals no output file was ever wrong and no wrong payload passed the inner checks in 30.2 million decode attempts; because those retrievals share 280 synthesis replicates, we report both the per-retrieval upper limit (1.6×10^{-4}) and the conservative per-synthesis one (1.3×10^{-2}). A leak-free replay of real Illumina streams from two vendor pools agrees on the ranking, and adds the one case where allocation pays substantially: a prior calibrated on held-out reads of the same containers, which identifies the weak container before retrieval, saved 16%. Every number is regenerated from committed result files under frozen protocols with logged deviations.**

***Index Terms*—DNA data storage, adaptive sequencing, verified stopping, erasure coding, coverage depth, digital twin, Bayesian calibration, multi-armed bandits, reproducibility**

I. Introduction

DNA has been demonstrated as an archival medium at kilobyte to hundreds-of-megabyte scale [1]–[5]. Retrieval requires sequencing, which samples molecules at random: some strands are read many times, others never, and reads carry substitution, insertion and deletion errors [6], [7]. Systems therefore protect each strand with an inner code and groups of strands with an outer erasure code, and sequence to a fixed depth expected to recover enough strands [3], [5]. The number of reads needed for decoding—the *coverage depth*—is now studied in its own right [8], [9].

We consider archives split across several physically separate *containers* (vials, capsules, pools) that are synthesised, stored and handled separately. This is a working assumption rather than established practice; random access so far addresses files within one pool [5], [10], and recent theory treats erasure-coded data partitioned over containers with uniform sampling [11] and container failures [12]. Containers age, are amplified and fail differently, and these differences are unknown when a retrieval starts. Two operational questions follow: *when* to stop sequencing, and *which* container to sequence next.

Problem. Given an erasure-coded file distributed over M containers of unknown, heterogeneous quality, some of which may have failed, sequence in batches and decide after each batch which container to read and whether to stop, so that the output file is byte-identical to the stored one with high probability while the number of reads is small.

Findings. (i) *Verified stopping is the robust saving, and it is a consumed-versus-reserved saving.* It needs no model, never returned a wrong file, and converts the 4.80–5.15k reads that a static plan must reserve into a 3.49k average bill on the digital twin (-27 to -32%); on both real-read pools it cut reads against fixed depth by 14% and 25%. (ii) *Allocation is worth a few percent at best, and is harmful under a misspecified container model.* With one efficiency multiplier per container every model-based rule read more than round-robin when a container was near-dead; with efficiency *and* dropout the best rule (UCB) became 3.5% cheaper than round-robin, and a hindsight static plan with the same stopping rule was 4.2% cheaper. Allocation pays substantially only when the weak container is identified *before* retrieval, from calibration reads (-16% on one real-read pool). (iii) *Nothing we tried lowered the 0.99-quantile*, so none of this reduces the capacity a user must reserve; the gains are mean-only. (iv) *Two zero-success*

results mark boundaries of the archive design, not of the policies: two failed containers leave exactly K fragments per stripe by construction, and under the photolithographic synthesis model no policy or static plan in our design recovered the file.

Scope. All work is computational. Nothing was synthesised or sequenced for this paper, and no result should be read as a wet-lab demonstration. We use the term *exact retrieval* only for our digital-twin pipeline, where the output file is checked byte for byte against the stored file; on replayed real reads, decodability is an oracle event and we speak of *recovery*.

Contributions.

1) *A measurement of what verified stopping is worth, with the comparison stated correctly.* A verification-gated streaming pipeline (Section IV) evaluated against a static family that includes a hindsight-tuned plan and a calibration-estimated plan, separating reads consumed from capacity reserved (Section VI-B). Stopping when decodable [13] and checksum-verified decoding [4] are both known; quantifying their value for heterogeneous multi-container retrieval, end to end to verified bytes, is what we add.
2) *A negative result for adaptive allocation, with its cause isolated and repaired.* Eight rules, including Thompson sampling, UCB, learn-then-commit and information-greedy design [14]–[17], lose to round-robin under a near-dead container; a $2\times2\times2$ ablation separates the container model from the likelihood and the prior (Section VI-E); and a two-parameter model with dropout removes most of the penalty (Section VI-F), which shows the failure is the model's, not adaptivity's, and bounds what a better model buys.
3) *Leak-free, end-to-end reproducibility, with honest uncertainty.* The full chain from DT4DDS reads [7] of independently synthesised containers to verified bytes under pre-registered protocols; safety bounds reported both per retrieval and per synthesis replicate; tail metrics with cluster-bootstrap intervals; sensitivity to the prior, the consensus cap, six further archive geometries and five failure modes; a cost model in which reads are not the only charge; and an artifact from which every table and figure is regenerated (Section IX).

II. Background and Related Work

DNA storage systems and channels. Storage systems combine constrained encodings with inner and outer codes [2]–[5], [18], [19]. Coverage is over-dispersed, a few percent of strands drop out, and errors arise mostly in synthesis and sequencing [6], [7], [20]. DT4DDS [7] is a digital twin of the synthesis, aging, PCR and sequencing steps whose error and bias parameters were fitted to experiments; we use it as the simulator of our primary evidence. Information-theoretic models treat the channel as sampling with replacement followed by noise [21]–[23].

Coverage depth. For a sampler that cannot be controlled, MDS codes and uniform sampling minimise the expected number of reads [8]. Follow-ups study random access [9], real-data sampling distributions [24], single-container coverage depth of structured codes [25], and the reliability of MDS-coded retrieval [26].

Distributed coverage depth. Two 2026 papers are closest to our setting and we state the relation precisely. Kong *et al.* [11] define the distributed coverage-depth problem: the n columns of a rank-k generator matrix are split over M containers by an ordered partition, and in every round *one* column is drawn uniformly at random, with replacement and *noiselessly*, from *each* container; $\tau(G,\mathcal{P})$ is the first round at which the drawn columns span $\mathbb{F}_q^k$. They derive an exact formula for $\mathbb{E}[\tau(G,\mathcal{P})]$ for arbitrary linear codes and partitions by q-binomial Möbius inversion, prove that an $[n,k]_q$ MDS code is optimal for every fixed partition (in rounds and in total reads), give the lower bound $M\,\mathbb{E}[\tau] \geq M\lceil k/M\rceil$ with $\min_{M,\mathcal{P}} M\,\mathbb{E}[\tau] = k$ attained by a staircase k-partition, characterise one-round-perfect layouts, separate the regimes in which distribution genuinely lowers total reads (constant-size containers) from those in which it only buys parallelism (containers of size k^{ε}), and prove that the q-ary simplex code uniquely minimises single-container coverage depth, resolving a conjecture of Bertuzzo *et al.* [25]. Their model is deliberately idealised: sampling is uniform within and across containers, containers are homogeneous and never fail, reads are noiseless, every round costs M reads whether or not they are needed (their Remark II.1), and the allocation is fixed in advance. Their Section V lists "nonuniform, asynchronous, adaptive, and noisy sampling" as open problem 4 and an asynchronous variant with a total read budget as an open problem. Our work is an empirical study inside exactly that gap — heterogeneous, noisy, possibly failed containers, reads allocated one batch at a time, and stopping decided by verification rather than by an oracle span condition — and it is complementary, not competing: we prove nothing about optimal codes or layouts, and we use their regime distinction (parallelism versus genuine read saving) to interpret our results. Levy *et al.* [12] study a different objective: reconstructing the contents of one *failed* container from the surviving ones, with scalar MDS and MDS array codes. Full-message retrieval, which we study, is still required after such a repair, and the two stopping rules differ (a prescribed missing block versus the whole message). Neither work allocates reads adaptively, uses noisy reads, or decodes to bytes. These works analyse *fixed* sampling laws; we study a *controllable* sampler empirically.

Adaptive sequencing and sequential decisions. Nanopore adaptive sampling selects molecules in real time [27]–[29], and DNA-storage readouts have been sequenced until sufficient coverage [13]. Choosing a container resembles a multi-armed bandit with a combinatorial (K-of-N per stripe) reward; we use Thompson sampling [14], [30], upper confidence bounds [15], explore-then-commit [16] and an information-greedy rule from Bayesian experimental design [17] as baselines (see [31]). We prove no regret bound and no optimality result: this is an experimental paper, and its claims are empirical claims about the configurations we tested.

Placement and consensus. Erasure-coded storage limits the fragments of a stripe per failure domain [32]–[36]. Strands are reconstructed from noisy reads by trace reconstruction [37]–[39] after clustering [40] or learned models [41]; systematic nanopore errors are documented [42], [43]. Table I positions this work.

What is and is not new here. (i) Prior DNA-storage work already stops sequencing when the decoder has enough information [13] and already verifies decoded data with checksums [4]; the coverage-depth literature already analyses the expected number of reads until decodability, including across containers [8], [11], [12]. We do not claim to introduce stopping, verification, or distributed DNA read allocation. (ii) What we did not find in that literature is a measurement, on an end-to-end pipeline that decodes to verified bytes, of how much a verification-gated stop is worth relative to the capacity a static plan must reserve when containers are heterogeneous and may have failed, and of whether choosing *which* container to read next with a learned container model helps or hurts under those conditions. (iii) Our contribution is that measurement, the mechanism behind the negative allocation result, and an artifact from which every number can be regenerated.

III. System Model

Table II lists the notation.

A. Storage

A file is bundled, hashed (SHA-256) and split into S stripes of K data fragments of P bytes. A systematic Cauchy Reed–Solomon code [46] over GF(2^8) extends each stripe to N fragments, any K of which suffice. Each fragment becomes a strand carrying an address, an address check byte, the payload and an inner Reed–Solomon code with four parity bytes plus a CRC-16, mapped to DNA by a whitened 2-bit code. Code parameters, a CRC-32 per stripe and the SHA-256 are kept in a manifest outside the DNA. Fragments are assigned round-robin to M containers.

B. Sequencing and recovery model used by the controller

Reads from container j form a stream. The controller's model (used only to rank containers and to forecast; never to decide that a fragment is recovered) treats the reads of a stored fragment as negative-binomial with mean $\theta_j\, R_j\, \bar{\pi}_j$ after R_j reads, where $\bar{\pi}_j$ is the prior per-fragment read share (yield over stored fragments) and θ_j an unknown efficiency multiplier (θ_j=0: failed container). A fragment becomes decodable after t reads with probability $\varphi(t)$ (consensus-success curve), and a molecule is absent with probability d (dropout). Integrating over abundance gives the probability $Q(\mu)$ that a stored fragment is decodable at mean coverage μ:

$$Q(\mu) = (1-d)\,\mathbb{E}_g\Big[\sum_{n\geq 0} \mathrm{NB}(n; k, g\mu)\,\varphi(n)\Big], \qquad (1)$$

with a shared log-normal strand effect g (Gauss–Hermite quadrature). Every container starts from the same generic literature prior (yield 0.9, NB shape k=10, dropout 0.5%, a generic consensus curve, failure probability 10^{-3}); no calibration data of the evaluated system is used in the primary experiments.

C. Objective and metrics

A policy chooses, after each batch of b reads, a container and whether to stop. Several distinct quantities are easy to conflate, so we define and report all of them separately, and we never compare two of them as if they were one:

1) **Mean reads over all attempts** $\bar{r}_{\text{all}}$: the average reads consumed per retrieval *attempt*, counting attempts that end in failure (which consume the whole stream). This is what a pay-per-read user pays on average.
2) **Mean reads conditional on success** $\bar{r}_{\text{ok}}$: the average over attempts that returned the verified file. It is the right metric only when the success probability is the same for the policies compared, and it says nothing about the cost of failures.
3) **Success probability** p_{ok}: the fraction of attempts that returned the verified file within the stream.
4) **Tail quantiles** q95 and q99 of the consumed reads, with cluster-bootstrap intervals.
5) **Reserved capacity** $B_{0.99}$: the budget a plan must commit *in advance* to succeed on 99% of retrievals, i.e. the q99 of its read requirement. A user who reserves a flowcell pays this on every retrieval, whether or not the retrieval needed it.

Verified stopping converts the reserved capacity $B_{0.99}$ into the consumed mean $\bar{r}_{\text{all}}$: that is the saving it offers, and it is a comparison between a policy that stops and a plan that does not. Comparing two policies that both stop is a separate, allocation-only question, and we report it separately (Section VI-B). No metric here is a monetary cost; a cost model with per-run and per-container charges is in Section **??**.

IV. Online-Adaptive Retrieval

A. Posterior over container efficiency

The support of θ_j is $\Theta = \{0\} \cup \{\theta^{(1)}, \ldots, \theta^{(25)}\}$, log-spaced in $[0.01, 5]$, with prior $p_0(0) = f_j$ and $p_0(\theta^{(\ell)}) \propto (1-f_j)\exp(-\frac{1}{2}(\ln\theta^{(\ell)}/s)^2)$, with $s = 0.5$ in every policy run. After R_j reads with U_j verified fragments among the n_j stored in container j, we use a beta-binomial likelihood with intra-container correlation ρ_U:

$$p(\theta \mid R_j, U_j) \propto p_0(\theta)\,\mathrm{BB}\big(U_j;\, n_j,\, Q(\theta R_j \bar{\pi}_j),\, \rho_U\big), \qquad (2)$$

where $\mathrm{BB}(u; n, p, \rho)$ has mean np and variance $np(1-p)(1+(n-1)\rho)$; ρ_U=0 recovers the binomial. Recoveries of a container's fragments are correlated (shared synthesis bias, dropout), so the binomial is over-confident; ρ_U is chosen on disjoint calibration replicates (Section V-D). We call the controller *online-adaptive*, not self-calibrating: it adapts to observations, but its uncertainty is not well calibrated (Section VI-J).

TABLE I

POSITIONING AGAINST THE CLOSEST PRIOR WORK. Y: YES; N: NO; P: PARTIAL; I: IMPLICIT (EXPECTED READS UNTIL DECODABILITY ANALYSED, NO OPERATIONAL STOPPING RULE); –: NOT STATED. "CONTAINERS" ARE STORAGE NODES IN [36] AND SAMPLING SOURCES IN [44].

Work	Multi-container	Container failures	Heterog. containers	Correlated errors	Adaptive read allocation	Stopping rule	Real reads	End-to-end digital twin
Lopez et al. 2019 [13]	N	N	N	–	N	P	Y	N
Gimpel et al. 2023 [7]	N	N	P	N	N	N	Y	Y
Bar-Lev et al. 2025 [8]	N	N	N	N	N	I	N	N
Gruica et al. 2024 [9]	N	N	N	N	N	I	N	N
Cao & Chen 2025 [24]	N	N	N	N	N	I	P	N
Kas Hanna 2026 [26]	N	N	N	N	N	N	N	N
Levy et al. 2026 [12]	Y	Y	N	N	N	I	N	N
Kong et al. 2026 [11]	Y	N	N	N	N	I	N	N
Leong et al. 2012 [36]	Y	Y	N	N	N	N	N	N
Weilguny et al. 2023 [29]	N	N	–	N	Y	–	Y	N
Bubeck et al. 2013 [44]	Y	N	Y	N	Y	N	N	N
This work	Y	P	Y	P	Y	Y	P	Y

Notes on "this work": container failures are simulated (digital twin) or injected (replay), not observed; correlated errors are validated on real nanopore reads but enter the controller only through consensus curves; real reads (ENA PRJEB65931 [45], same-batch aliquots) are used in a semi-empirical replay with oracle decodability, so they are marked partial; the end-to-end digital twin is DT4DDS [7]. Gimpel et al. [7]: heterogeneity characterised across handling conditions of one pool, and the digital twin itself. Cao & Chen [24]: log-normal channel fitted to their own data. Weilguny et al. [29]: genomic loci, not storage containers. Kong et al. [11] (full paper, 33 pp.): containers are homogeneous, never fail and are sampled noiselessly and uniformly, one read per container per round, with the allocation fixed in advance; adaptive, nonuniform, asynchronous and noisy sampling are listed there as open problems. Levy et al. [12]: objective is repairing one failed container from the survivors, not full-message retrieval. Neither derives an operational stopping rule, so both are marked I.

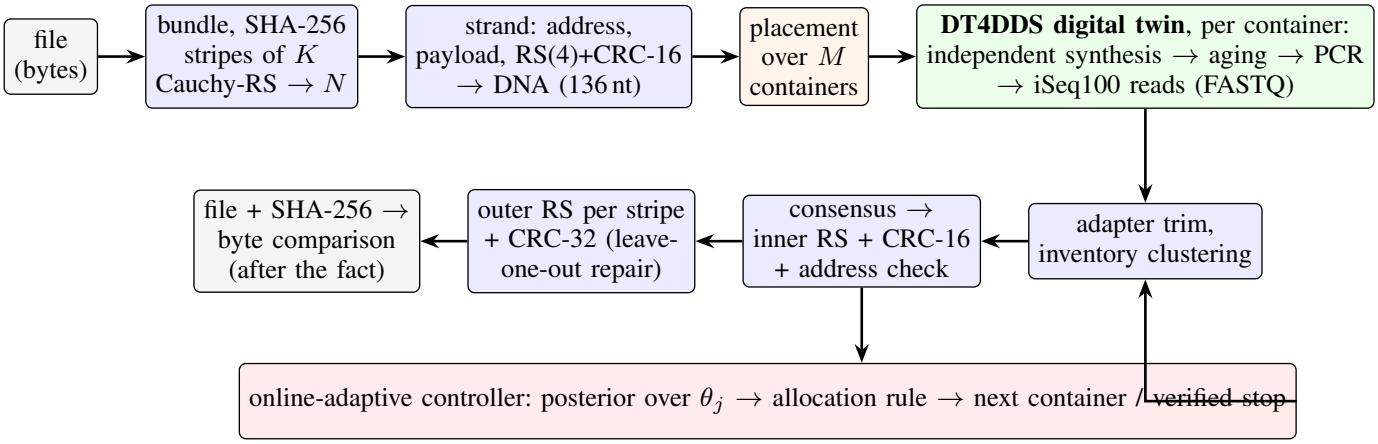


Fig. 1. Pipeline of the primary experiments (conceptual diagram). Every container is an independent DT4DDS synthesis with its own handling history; its reads are consumed in batches by the same streaming decoder that reads FASTQ files. The true file is used only after a retrieval ends, to count wrong outputs and false accepts.

B. Allocation rules

For an unrecovered fragment i in container j, the hazard of becoming decodable within the next b reads is $h_{ij} = [Q(\tilde{\theta}_j(R_j+b)\bar{\pi}_j) - Q(\tilde{\theta}_j R_j \bar{\pi}_j)]/[1 - Q(\tilde{\theta}_j R_j \bar{\pi}_j)]$, where $\tilde{\theta}_j$ is chosen by the rule:

- *Round-robin + verified stop*: reads every container in turn (no model).
- *Coverage-greedy*: $\tilde{\theta}_j$ = posterior mean; reads $\arg\max_j \sum_i h_{ij}$ over fragments of unfinished stripes.
- *Pivotal*: weights each fragment by a normal-approximation sensitivity of its stripe's completion probability (our earlier rule; details in the supplement).
- *Thompson sampling* [14], [30]: $\tilde{\theta}_j$ drawn from the posterior; coverage score.
- *UCB* [15]: $\tilde{\theta}_j$ = posterior 0.9-quantile.
- *Learn-then-commit* [16]: five round-robin batches per container, then the posterior is frozen and the coverage score used.
- *Info-greedy* (Bayesian experimental design [17]): reads the container with the largest posterior variance of $\ln \theta_j$ until all fall below 10^{-3}, then coverage-greedy.
- *Tail-aware*: $\tilde{\theta}_j$ = posterior 0.1-quantile (pessimistic) and the score counts only the 10% of stripes with the largest remaining need (the bottleneck stripes that set the retrieval's length); a heuristic aimed at the q99.

All rules probe each container with one batch first. Table III gives the operational specification of the loop and of every rule — initialisation, batch size, tie-breaking, when the posterior is updated, the stopping test and how an exhausted stream is handled — so that nothing has to be read out of the code. The *static* family reads fixed proportions $w_j \propto e_j^{\alpha}$, $\alpha \in \{0, 0.5, 1, 2\}$, until decodable, where e_j is the per-read

TABLE II
NOTATION (CODE OBJECTS IN PACKAGE D3RA).

Symbol	Meaning	Code
K, N, S, P	data / total fragments per stripe, stripes, payload bytes	`erasure_codes`
M, j	number of containers, container index	`Layout`
n_j	fragments stored in container j	`Layout`
b	batch size (reads per decision)	`AdaptiveConfig`
R_j, U_j	reads taken from j, verified fragments from j	`StreamRetriever`
θ_j, Θ, s	efficiency multiplier, its grid, prior log-sd	`AdaptiveConfig`
f_j	prior probability that container j failed	`ChannelParams`
$\bar{\pi}_j$	prior per-fragment read share of container j	`Adaptive Controller`
k, d, $\varphi(t)$	NB shape, dropout, consensus-success curve	`ChannelParams`
$Q(\mu)$	P(fragment decodable) at mean coverage μ	`_QTable`
ρ_U	beta-binomial ICC of recoveries in the likelihood	`overdispersion`
h_{ij}	hazard of recovery within the next batch	`scores`
q	posterior quantile for UCB (q) and tail-aware ($1-q$)	`quantile`
ν_s	remaining need of stripe s (K minus verified)	`scores`
w_j, e_j, α	static proportions, hindsight per-read recovery rate, exponent	`static_family`
θ^*_j	calibration target (best-fitting multiplier)	`_theta_star`
q99	0.99-quantile of reads per retrieval	`dt_analysis`

recovery rate of container j measured in hindsight on the same scenario: α=0 is uniform fixed depth, α=1 equalises expected coverage, and the best α is a hindsight-tuned ("oracle") static plan.

C. What a "container switch" means here, and what it would mean in a laboratory

Our controller picks a container after every batch of b=50 reads. We do not claim that a sequencer can be redirected between physical vials that often, and the paper should not be read as a demonstration of a physical sequencing controller. What our experiments model is an *allocation policy over M independent read streams*: each container has its own prepared library and its own stream of reads, and the policy decides from which stream the next batch is consumed. Three physical arrangements realise that abstraction with different costs. (i) *Multiplexed run*: all M libraries are barcoded and loaded on one flow cell; reads arrive interleaved and "choosing a container" means choosing which barcode's reads to keep processing. This matches our model closely but does not reduce the reads the instrument actually produces, so it saves analysis and, on pay-per-read services, money, but not instrument time. (ii) *Streaming (nanopore)*: reads arrive in real time, and adaptive sampling can reject molecules mid-read [27], [28]; with one flow cell per container, stopping is directly actionable, and with barcoded libraries selective sequencing approximates container choice. (iii) *Sequential runs*: one run per container, where a switch costs a library preparation and a run setup. Our frozen cost metric (reads) describes case (i) with per-read billing and is optimistic for cases (ii) and (iii); the per-batch and per-container charges in Section **??** bound how much that optimism matters. Verified stopping, the result we actually recommend, needs no switching mechanism at all: it only needs the decoder to be able to say "stop".

D. Verified stopping

A retrieval stops with an output file only when (i) every stripe holds K fragments that each passed inner RS decoding, CRC-16 and the address check, (ii) every stripe matches its CRC-32, and (iii) the file matches its SHA-256. If (ii) fails, one more fragment is requested for that stripe and the faulty fragment is localised by leave-one-out decoding. If the streams are exhausted first, the retrieval reports failure; it never returns an unverified file. An early-abandonment (futility) rule is disabled in all runs.

E. Completion forecast

The controller also forecasts the probability of finishing within a remaining budget. The *analytic* forecast multiplies per-stripe normal approximations of $\mathbb{P}(\sum_{i\in s} X_i \geq \nu_s)$ with the remaining budget spread evenly over containers. The *posterior-simulation* forecast (default) draws 200 values of θ from the posterior, spreads the remaining budget in proportion to the reads spent so far, simulates per-fragment recoveries and reports the fraction of draws in which every stripe completes.

V. EVALUATION METHODOLOGY

A. Digital-twin end-to-end pipeline (primary evidence)

We store a fixed 18,432-byte random file with P=24-byte payloads, $(K, N) = (32, 48)$, over M=6 containers. The stripe count follows from what is actually encoded: the library bundles the file with a 162-byte header (name, length, hash) into an 18,594-byte blob, and $\lceil 18{,}594/(32\times 24)\rceil = 25$ stripes of 32 data fragments cover it, the last one zero-padded (24 stripes would hold only 18,432 bytes, which is the payload alone). This gives $25 \times 48 = 1{,}200$ strands of 136 nt, 200 per container, 8 fragments of every stripe per container, and, at 60 simulated reads per stored strand, a maximum stream of $60 \times 1{,}200 = 72{,}000$ reads. Every count in this paper is the one the code produces, not a rounded design target. For each container we run DT4DDS 1.1.0 [7] with its published default parameters: array synthesis (Twist model; log-normal oligo counts, σ=0.3), sampling of a given number of copies per strand, aging, high-fidelity PCR and Illumina iSeq100 single-end 150 nt sequencing, producing 60 reads per stored strand. The six containers are *independent* syntheses with their own seeds and histories. The reads, after adapter trimming, go through inventory clustering against the expected address prefixes, consensus (at most 24 reads per cluster), inner decoding, outer decoding with CRC-32 repair and SHA-256 verification, in batches of 50 reads. Every container decodes its own clusters; nothing uses the stored strands to decide recovery. The stored file and payloads are used only after the retrieval to count wrong outputs and false accepts (a wrong payload that passed the inner checks).

TABLE III

OPERATIONAL SPECIFICATION OF THE RETRIEVAL LOOP AND OF EVERY ALLOCATION RULE. ALL RULES SHARE THE LOOP, THE BATCH SIZE, THE PROBE, THE TIE-BREAKING AND THE STOPPING TEST; THEY DIFFER ONLY IN THE SCORE σ_j AND IN WHICH POINT VALUE $\tilde{\theta}_j$ OF THE POSTERIOR THEY USE. R_j: READS SPENT ON CONTAINER j; U_j: FRAGMENTS OF CONTAINER j VERIFIED FROM ITS OWN READS; n_j: FRAGMENTS STORED IN j; h_{ij}: HAZARD THAT UNRECOVERED FRAGMENT i IN j BECOMES DECODABLE WITHIN THE NEXT BATCH; $\mathcal{O}$: FRAGMENTS OF STRIPES THAT STILL NEED FRAGMENTS.

Retrieval loop (identical for every policy)

1) $R \leftarrow 0$; pools empty; tracker empty.
2) *Probe*: read b=50 reads from every container that stores fragments, in index order (the static plans skip the probe and start from their proportions).
3) **repeat**
   a) *Stopping test*: if every stripe holds K fragments that passed inner RS, CRC-16 and the address check, outer-decode all stripes; if every stripe matches its CRC-32 and the blob matches its SHA-256, **return** the file. If a stripe fails its CRC-32, raise that stripe's requirement by one fragment (leave-one-out localisation) and continue; after 20 such rounds, stop and report failure.
   b) If every stream is exhausted, **return** failure (never an unverified file).
   c) Compute the posterior (2) from (R_j, U_j) for every container, take $\tilde{\theta}_j$ as the rule prescribes, score the available containers and read b reads from $\arg\max_j \sigma_j$; ties break to the lowest index.
   d) Cluster the new reads against expected address prefixes, form a consensus from at most 24 reads per cluster, decode, and update U_j and the tracker. Reads are charged even if the stream runs dry.

Allocation rules: score σ_j

- *Round-robin + verified stop*: σ_j unused; container (round $\bmod M$). No model is evaluated.
- *Static* α: no model; read the container minimising R_j/w_j with $w_j \propto e_j^{\alpha}$ (exhausted containers excluded; if all positive-weight containers are exhausted, read the rest so the failure is charged).
- *Coverage-greedy*: $\tilde{\theta}_j = \mathbb{E}[\theta_j]$; $\sigma_j = \sum_{i \in \mathcal{O} \cap j} h_{ij}$.
- *Pivotal*: $\tilde{\theta}_j = \mathbb{E}[\theta_j]$; $\sigma_j = \sum_i w_s(i)\, h_{ij}(1 - \rho_i + h_{ij})$ with w_s a normal-approximation sensitivity of stripe s's completion probability.
- *Thompson sampling*: $\tilde{\theta}_j \sim p(\theta_j \mid R_j, U_j)$, redrawn every round; coverage score.
- *UCB*: $\tilde{\theta}_j =$ posterior 0.9-quantile; coverage score.
- *Learn-then-commit*: round-robin for 5 batches per container, then freeze the posterior and use the coverage score with $\tilde{\theta}_j = \mathbb{E}[\theta_j]$ from the frozen posterior.
- *Info-greedy*: $\sigma_j = \mathrm{Var}[\ln \theta_j]$ while any container has $\mathrm{Var}[\ln \theta_j] > 10^{-3}$, then coverage-greedy. Containers with no unrecovered fragment score 0.
- *Tail-aware*: $\tilde{\theta}_j =$ posterior 0.1-quantile; score counts only fragments of the 10% of unfinished stripes with the smallest standardised slack, falling back to the coverage score when no such fragment can be advanced.

Container model. Frozen: $\theta_j \in \{0\} \cup \{0.01 \ldots 5\}$ (25 log-spaced points), prior $p_0(0) = 10^{-3}$ and log-normal with s=0.5 otherwise, beta-binomial likelihood with ρ_U=0.1. Revision 4 adds the two-parameter model: $d_j \in \{0, 0.05, 0.15, 0.3, 0.5, 0.7, 0.9\}$ with $p_0(d_j{=}0) = 0.6$ and the rest uniform, a joint posterior over (θ_j, d_j), verified-fragment probability $(1 - d_j)Q(\theta_j R_j \bar{\pi}_j)$, and hazards multiplied by $(1 - \hat{d}_j)$ so that a fragment still missing after many reads is increasingly attributed to permanent dropout.

B. Scenarios

Baseline: fresh; aged one half-life; aged two half-lives with 10 copies per strand; 30 PCR cycles; 45 PCR cycles with 20 copies; near-dead (two half-lives, 40 cycles, σ=0.5, 3 copies). *One/two failed*: the fresh (and the 30-cycle) container returns only non-informative reads. *Homogeneous control*: six fresh containers. *Photolithographic*: the baseline histories with DT4DDS's photolithographic synthesis model. *Severity sweep*: the baseline with the near-dead container's copy number set to 200, 30, 10, 3 or 1.

C. Frozen protocol and statistics

Design, policies, metrics, statistics, pre-specified comparisons and the calibration rule were written to `dt_protocol.yaml` before the final runs; deviations are listed in Appendix B. Each scenario uses 30 independent syntheses of all six containers $\times$ 10 read orders (300 retrievals per policy); seeds are derived with CRC-32 of the scenario, replicate and container, so every run is bit-for-bit reproducible. CIs are cluster bootstrap percentile intervals over syntheses (2,000 resamples [47]); comparisons are paired over the same syntheses and read orders; wrong-output and false-accept rates carry exact Clopper–Pearson bounds [48]. We report every pre-specified comparison, including negative ones, and no significance stars.

D. Calibration protocol

ρ_U is chosen from $\{0, 0.003, 0.01, 0.03, 0.1\}$ on ten *disjoint* baseline replicates (seeds 1000–1009) as the value whose 90% credible interval covers the target θ_j^* closest to 90% of the time. θ_j^* is the multiplier that best fits the container's full recovery curve under the generic model, and the selection used prior sd s=1 whereas the policy runs use s=0.5 (a mismatch we noticed after freezing and report rather than re-tune); intervals are evaluated at the read count where the prior predicts 40% recovery and at twice that. The selected value is then fixed as the default for every policy run, evaluated on the held-out replicates 0–29, and used on the real-read replay. The completion forecast is evaluated with a budget equal to the median coverage-greedy cost, at 30/50/70/85% of it, by expected calibration error (ECE, 10 bins), Brier score against a constant (climatology) forecast [49] and AUC.

E. Real read streams (secondary evidence)

We replay public Illumina iSeq runs of two pools from ENA PRJEB65931 [7], [45]: a Twist pool after 0/2/7 days of aging or 15/60/90 PCR cycles, and a Genscript pool (aging 0 d A/B, 2 d, 7 d; PCR 15/30 cycles). Each run emulates one container holding 250 stripes of $(32, 48)$, i.e. $250 \times 48 = 12{,}000$ designed strands. The Twist design file contains exactly 12,000 strands (108 nt), all of which are used. The Genscript design file contains 12,472 strands (102 nt); we draw a seeded random subset of 12,000 and *exclude* the remaining 472 designs

entirely, discarding the reads that map to them: of the $6 \times 900{,}000$ evaluation reads, 86.3% map to a design and 4.23% of those mapped reads are discarded as excluded designs, leaving 4,461,804 usable reads. The excluded strands serve no other purpose, and the subset is the same in every condition and policy, so all Genscript comparisons use identical read material. A fragment counts as recovered when the consensus of its first t reads in a random permutation of the run equals the designed strand, with t drawn from per-strand consensus curves estimated from real reads; decodability is therefore an *oracle* event and the replay is semi-empirical (read orders and counts real, containers emulated by subsetting runs of one synthesis batch). 200 read permutations, paired bootstrap.

Leakage audit of the replay. Each run is read once and split by position: the first 300,000 reads are the calibration split and the next 900,000 the evaluation split, so no read is in both. Everything that generates a recovery event in the evaluation comes from the evaluation split alone: the read order is a permutation of the evaluation reads, and the number of reads a strand needs is drawn against per-strand consensus curves. Two beliefs are used, and they differ exactly in which split they may see. The *generic* prior (all primary replay results) sees no data from these pools at all: yield 0.9, NB shape 10, dropout 0.5%, a generic consensus curve. The *leak-free calibrated* belief sees only the calibration split — per-container read counts and pooled consensus statistics estimated from calibration reads — and never the evaluation split. The one place where evaluation data is used deliberately is the hindsight-tuned static plan, which is labelled as an oracle baseline wherever it appears and exists to be an upper bound on what a static allocation could achieve. Our first analysis of these pools violated this discipline (per-strand consensus curves estimated on the evaluation reads and static weights tuned on them); it is superseded, kept only as an upper bound in Appendix A, and no claim in this paper rests on it.

VI. Results

A. Safety of verified stopping

Table IV reports the safety record and, next to it, the dependence structure behind it. Across 23,160 end-to-end retrievals no output file differed from the stored file in a single byte, and in 30.2 million fragment decode attempts no wrong payload passed inner RS decoding, the CRC-16 and the address check. These retrievals are *not* independent experiments: they come from 280 distinct synthesis replicates, about 83 retrievals per replicate (10 read orders $\times$ several policies and models on the same simulated DNA), and the decode attempts inside one retrieval share the same reads. A Clopper–Pearson bound that treats 23,160 retrievals as Bernoulli trials would therefore overstate what has been shown. We report both ends: per retrieval the upper limit is 1.6×10^{-4}, and treating one synthesis replicate as one unit — the conservative reading, which discards the information in repeated read orders — it is 1.3×10^{-2}. The false-accept limits are 1.2×10^{-7} per attempt and 1.3×10^{-2} per synthesis replicate. The honest summary is therefore qualitative as well as numerical: in every configuration we ran, verification-gated stopping never returned a wrong file, and the bound that does not assume independence is of order 10^{-2} per synthesis, not 10^{-4}. A false accept would need a wrong payload to pass a 4-symbol RS code, a CRC-16 and an address byte, then the stripe CRC-32 and the file SHA-256; the cryptographic hash is the backstop, and no experiment of this size can bound its failure rate. Table IV pools every digital-twin retrieval run for this paper. No retrieval returned a file that differed from the stored one, and no wrong payload passed the inner RS + CRC-16 + address checks. The bounds are those of zero events in the given number of trials; they are statements about this archive geometry, these error rates and this code stack, not about DNA storage in general. Unsuccessful retrievals (one or two failed containers) ended by exhausting the streams and reported failure instead of returning a file.

TABLE IV
Safety of verification-gated stopping over every digital-twin retrieval of this paper, with the dependence between retrievals made explicit. Retrievals that share a synthesis replicate are not independent, so we also report the bound that treats one synthesis as one unit. Upper limits of exact two-sided 95% Clopper–Pearson intervals.

Quantity	Value
Retrievals (all scenarios, policies, geometries)	23,160
wrong output files	0
Independent synthesis replicates	280
retrievals per replicate (mean)	83
synthesis replicates with a wrong output	0
Wrong-output rate per retrieval, 95% upper limit	1.6×10^{-4}
per synthesis replicate (conservative)	1.3×10^{-2}
Fragment decode attempts	30,164,044
false accepts (wrong payload passed inner checks)	0
attempts per synthesis replicate (mean)	107,729
False-accept rate per attempt, 95% upper limit	1.2×10^{-7}
per synthesis replicate (conservative)	1.3×10^{-2}

B. Reads consumed versus capacity reserved

The largest effect in this paper is the difference between reads that are *consumed* and capacity that must be *reserved*, and it is easy to misattribute, so we separate it first (Table V). In the baseline digital twin, round-robin with verified stopping consumed 3.49k reads on average (95% CI 3.37–3.60k), while the same uniform read proportions without stopping must reserve 5.15k to succeed on 99% of retrievals, and the best static plan in the family must reserve 4.80k. Stopping converts a 4.80–5.15k reservation into a 3.49k average bill: a 27–32% reduction, depending on which plan the reservation is taken from, and it needs no model of the containers.

Allocation is a separate and much smaller effect. When the static plans run *with* the same verified stopping — the only fair policy-versus-policy comparison — the hindsight-tuned proportions $\alpha{=}1$ consume 3.34k reads, 4.2% *less* than round-robin's 3.49k, and the model-based rules consume 3.75–18.87k, 7–441% *more*. In the baseline the ranking is therefore: a hindsight static plan with stopping is best by a few percent, round-robin with stopping next, and every online rule we tried worse. Our earlier revision quoted the 27% figure in a way

TABLE V

READS *consumed* WITH VERIFIED STOPPING VERSUS CAPACITY THAT MUST BE *reserved* WITHOUT IT (DIGITAL TWIN, 300 RETRIEVALS PER ROW). ALL ROWS STOP ON VERIFICATION, SO THE COMPARISON BETWEEN ROWS IS ALLOCATION ONLY; THE LAST COLUMN IS THE BUDGET THE SAME READ PROPORTIONS WOULD NEED WITHOUT STOPPING. THOUSANDS OF READS; MEAN OVER ALL ATTEMPTS WITH A CLUSTER-BOOTSTRAP 95% INTERVAL.

Scen.	Allocation	p_{ok}	$\bar{r}_{\text{all}}$ [95% CI]	q95	q99	$B_{0.99}$
Base	Static α=0 (fixed depth)	1.00	3.49 [3.37, 3.60]	4.56	5.15	5.15
	Static α=0.5	1.00	3.40 [3.29, 3.49]	4.55	4.90	4.90
	Static α=1	1.00	3.34 [3.24, 3.44]	4.45	4.80	4.80
	Static α=2	1.00	3.36 [3.27, 3.44]	4.40	4.90	4.90
	Round-robin + verified stop	1.00	3.49 [3.37, 3.60]	4.56	5.15	5.15
	Coverage-greedy	1.00	4.02 [3.77, 4.30]	5.80	9.05	9.05
	Pivotal	1.00	5.21 [4.75, 5.70]	8.85	9.60	9.60
	Thompson sampling	1.00	3.75 [3.57, 3.94]	5.21	6.45	6.45
	UCB	1.00	4.02 [3.77, 4.30]	6.05	8.15	8.15
	Learn-then-commit	1.00	5.05 [4.74, 5.31]	6.80	7.65	7.65
	Info-greedy	1.00	18.87 [17.25, 20.31]	31.44	37.45	37.45
	Tail-aware	1.00	3.94 [3.71, 4.17]	5.65	7.15	7.15
Homog.	Static α=0 (fixed depth)	1.00	2.26 [2.23, 2.29]	2.60	2.85	2.85
	Static α=0.5	1.00	2.26 [2.24, 2.29]	2.65	2.85	2.85
	Static α=1	1.00	2.26 [2.24, 2.29]	2.65	2.85	2.85
	Static α=2	1.00	2.26 [2.24, 2.29]	2.65	2.85	2.85
	Round-robin + verified stop	1.00	2.26 [2.23, 2.29]	2.60	2.85	2.85
	Coverage-greedy	1.00	2.18 [2.16, 2.21]	2.50	2.70	2.70
	Pivotal	1.00	2.18 [2.16, 2.20]	2.50	2.75	2.75
	Thompson sampling	1.00	2.19 [2.17, 2.22]	2.50	2.70	2.70
	UCB	1.00	2.18 [2.16, 2.20]	2.50	2.75	2.75
	Learn-then-commit	1.00	2.19 [2.17, 2.21]	2.50	2.75	2.75
	Info-greedy	1.00	2.25 [2.23, 2.28]	2.60	2.90	2.90
	Tail-aware	1.00	2.20 [2.18, 2.23]	2.55	2.75	2.75

that could be read as an allocation result; it is not one, and the comparison it does make (consumed versus reserved) is the one that carries the recommendation of Section VII.

C. Policy comparison on the digital twin

Table VI and Fig. 2 give the pre-specified comparisons. In the baseline scenario every policy recovered the file in all 300 retrievals. Round-robin with verified stopping used 3.49k reads on average (95% CI 3.37–3.60k). Every model-based rule was costlier on the same syntheses and read orders: Thompson sampling by 7% (4–11%), tail-aware by 13% (9–18%), coverage-greedy by 15% (10–21%), UCB by 15% (11–20%), learn-then-commit by 45% (40–49%), pivotal by 49% (39–60%) and information-greedy by 441% (408–467%). The ordering follows the share of reads spent on the near-dead container: 16% for round-robin, 23% for Thompson sampling, 25–27% for tail-aware, UCB and coverage-greedy, 40–41% for pivotal and learn-then-commit, and 66% for information-greedy. In the homogeneous control (six fresh, independently synthesised containers), where we expected no allocation effect, most model-based rules were 2–4% cheaper than round-robin (coverage-greedy 2.18k vs. 2.26k; information-greedy even at 2.26k): independent syntheses differ enough for mild adaptive gains, consistent with the low-severity end of the sweep below. With DT4DDS's photolithographic synthesis model (reduced design, 30 retrievals per policy) no policy and none of the four static plans recovered the file: every retrieval read the whole 72k-read stream and reported failure, with no wrong output. These two zero-success results have different standing and we state them separately. For *two failed containers* the boundary is arithmetic: with $(32, 48)$ over six containers each container holds exactly 8 of the 48 fragments of every stripe, so losing two leaves exactly K=32 fragments and no margin for a single further strand loss; no policy and no read budget can recover the file, and this follows from the code and placement rather than from our experiments. For the *photolithographic* scenario the statement is empirical and narrower: under that synthesis model, with this code, this decoder, a 24-read consensus cap and a 72k-read stream, none of the eight policies and none of the four static plans recovered the file in any of the retrievals we ran, and no wrong file was returned. We did not test deeper streams, a stronger inner code, a larger consensus cap or a lower code rate, so this is a boundary of *our* archive design under that synthesis model, not a claim that photolithographic DNA archives are unrecoverable.

D. What the gain after a container failure buys

The one setting in which the online rules won is a container that failed outright, and that result needs its denominator stated (Table VII). With one failed container 34% of retrievals succeeded, and *the success probability was identical for every policy and every static plan*: whether the file can be recovered at all is decided by which fragments survive in the five remaining containers, not by the reading order. Among the 102 retrievals that succeeded, UCB used 15.49k reads against round-robin's 20.31k, 24% fewer. Counting all 300 attempts, including the 66% that read the whole 72k-read stream and failed, UCB used 52.79k against round-robin's 54.43k, a 3% reduction. Both numbers are correct and they answer different questions: a user who retries until success pays the conditional number, a user who pays per attempt pays the unconditional one. We therefore state the result as: *after an outright failure, posterior rules cut the cost of the retrievals that succeed by*

TABLE VI

DIGITAL-TWIN END-TO-END RETRIEVAL (DT4DDS; PRIMARY EVIDENCE). PER SCENARIO AND POLICY: SHARE OF RETRIEVALS WHOSE OUTPUT FILE IS BYTE-IDENTICAL TO THE STORED FILE (NO RETRIEVAL RETURNED A WRONG FILE), MEAN READS PER RETRIEVAL (THOUSANDS; CLUSTER-BOOTSTRAP 95% CI OVER SYNTHESES), THE 0.99-QUANTILE Q99 (THE BUDGET TO RESERVE), AND THE PAIRED RELATIVE DIFFERENCE IN MEAN READS AGAINST ROUND-ROBIN WITH VERIFIED STOPPING ON RETRIEVALS WHERE BOTH SUCCEEDED (95% CI). STATIC PLANS READ FIXED PROPORTIONS UNTIL DECODABLE AND MUST RESERVE THEIR Q99 ON EVERY RETRIEVAL, SO THEIR MEAN COST EQUALS Q99; THEIR PROPORTIONS USE PER-CONTAINER EFFICIENCIES MEASURED IN HINDSIGHT ON THE SAME SCENARIO.

Scenario	Policy	Success	Mean reads (k) [95% CI]	q99 (k)	vs. round-robin, % [95% CI]
Baseline	Round-robin + verified stop	1.00	3.49 [3.37, 3.60]	5.15	
	Coverage-greedy	1.00	4.02 [3.76, 4.30]	9.05	+15 [+10, +21]
	Pivotal	1.00	5.21 [4.76, 5.70]	9.60	+49 [+40, +60]
	Thompson sampling	1.00	3.75 [3.56, 3.95]	6.45	+7 [+4, +11]
	UCB (q=0.9)	1.00	4.02 [3.78, 4.30]	8.15	+15 [+11, +20]
	Learn-then-commit	1.00	5.05 [4.74, 5.31]	7.65	+45 [+40, +49]
	Info-greedy (exp. design)	1.00	18.87 [17.15, 20.37]	37.45	+441 [+407, +470]
	Tail-aware (LCB)	1.00	3.94 [3.72, 4.18]	7.15	+13 [+8, +18]
	Static, uniform (fixed depth)	1.00	5.15 (= q99)	5.15	
	Static, $w \propto e^{0.5}$	1.00	4.90 (= q99)	4.90	
	Static, $w \propto e$ (equal coverage)	1.00	4.80 (= q99)	4.80	
	Static, $w \propto e^2$	1.00	4.90 (= q99)	4.90	
One container failed	Round-robin + verified stop	0.34	20.31^s	–	
	Coverage-greedy	0.34	16.86^s	–	-17 [-23, -9]
	Pivotal	0.34	17.30^s	–	-15 [-21, -6]
	Thompson sampling	0.34	18.43^s	–	-9 [-15, -1]
	UCB (q=0.9)	0.34	15.49^s	–	-24 [-28, -17]
	Learn-then-commit	0.34	21.91^s	–	+8 [+3, +14]
	Info-greedy (exp. design)	0.34	36.68^s	–	+81 [+50, +130]
	Tail-aware (LCB)	0.34	20.63^s	–	+2 [-8, +14]
	Static, uniform (fixed depth)	0.34	–	–	
	Static, $w \propto e^{0.5}$	0.34	–	–	
	Static, $w \propto e$ (equal coverage)	0.34	–	–	
	Static, $w \propto e^2$	0.34	–	–	
Two containers failed	all 12 policies and static plans	0.00	– (stream exhausted, 72k reads)	–	
Homogeneous (control)	Round-robin + verified stop	1.00	2.26 [2.23, 2.29]	2.85	
	Coverage-greedy	1.00	2.18 [2.16, 2.21]	2.70	-3 [-4, -2]
	Pivotal	1.00	2.18 [2.16, 2.20]	2.75	-4 [-5, -3]
	Thompson sampling	1.00	2.19 [2.17, 2.22]	2.70	-3 [-4, -2]
	UCB (q=0.9)	1.00	2.18 [2.16, 2.20]	2.75	-4 [-5, -3]
	Learn-then-commit	1.00	2.19 [2.17, 2.21]	2.75	-3 [-4, -2]
	Info-greedy (exp. design)	1.00	2.25 [2.23, 2.28]	2.90	-0 [-1, +0]
	Tail-aware (LCB)	1.00	2.20 [2.18, 2.23]	2.75	-2 [-4, -1]
	Static, uniform (fixed depth)	1.00	2.85 (= q99)	2.85	
	Static, $w \propto e^{0.5}$	1.00	2.85 (= q99)	2.85	
	Static, $w \propto e$ (equal coverage)	1.00	2.85 (= q99)	2.85	
	Static, $w \propto e^2$	1.00	2.85 (= q99)	2.85	
Photolithographic synthesis‡	all 12 policies and static plans	0.00	– (stream exhausted, 72k reads)	–	

30 independent syntheses of all six containers $\times$ 10 read orders = 300 retrievals per policy and scenario, except ‡10 $\times$ 3 (protocol deviation, compute). A failed retrieval read the whole stream (72,000 reads) without verifying; ^{s}: mean over successful retrievals only (a mean or quantile including failures is undefined, "–"). When success < 1, success was identical for every policy on every synthesis and read order, so the paired comparison uses the same retrievals for all policies.

up to 24%, without improving the probability of success, and with a 3% effect on the cost per attempt.

E. Why adaptive allocation loses, and what fixes it

The frozen controller models a container by one efficiency multiplier θ_j that scales every fragment's coverage. The near-dead container (two half-lives, 40 PCR cycles, σ=0.5, three copies per strand) instead loses a large fraction of its molecules outright: many of its fragments are never read however long it is sequenced, while the surviving ones are read often. Early batches from it therefore recover fragments at a rate the model explains with a moderate θ_j and extrapolates to the missing fragments, so coverage-based scores keep predicting gains there. The beta-binomial likelihood (ρ_U=0.1) widens the posterior, which is what calibration asked for, and slows the correction further; information-greedy, which reads the container with the largest posterior variance, never brings that variance below its threshold and spends two thirds of its reads there.

Separating the three candidate causes. Three things could be responsible: the container model, the likelihood (ρ_U), and the prior width (s). We ran the $2 \times 2 \times 2$ ablation of Table **??** to tell them apart, rather than attributing the result to the model by argument. The likelihood matters: with the binomial likelihood (ρ_U=0) on the same retrievals, coverage-greedy, Thompson sampling and tail-aware are 3–4% *cheaper* than round-robin instead of 7–15% costlier, so the better-calibrated posterior is the costlier one. The prior width matters much less. But the container model dominates, and it is the one factor that changes the conclusion.

F. An efficiency-and-dropout container model

Representing a container by efficiency *and* a dropout fraction — a joint posterior over (θ_j, d_j) with $d_j \in$

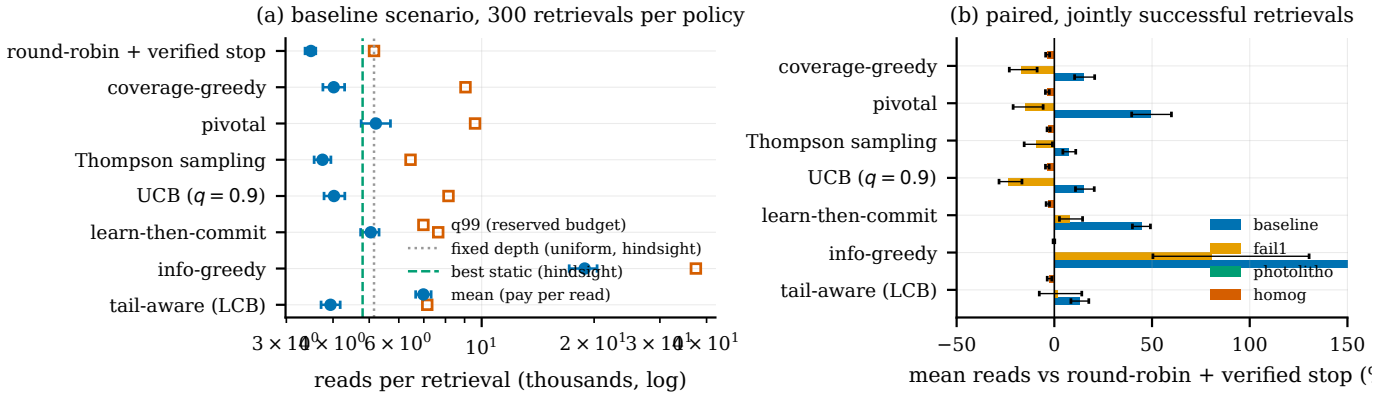


Fig. 2. Digital-twin end-to-end retrieval. (a) Baseline scenario: mean reads (95% cluster-bootstrap CI) and q99 per policy; vertical lines: static plans (whose cost equals their q99). (b) Paired relative cost against round-robin with verified stopping on retrievals where both succeeded (95% CI); negative is cheaper.

TABLE VII

ONE FAILED CONTAINER: COST CONDITIONAL ON SUCCESS AND OVER ALL ATTEMPTS (300 RETRIEVALS PER ROW, 72K-READ STREAM). EVERY POLICY HAS THE SAME SUCCESS PROBABILITY, SO THE CONDITIONAL SAVING DOES NOT BUY A BETTER CHANCE OF RECOVERY, AND FAILURES DOMINATE THE UNCONDITIONAL MEAN. THOUSANDS OF READS.

Allocation	p_{ok}	$\bar{r}_{\text{ok}}$	vs. RR	$\bar{r}_{\text{all}}$
Round-robin + verified stop	0.34	20.31	–	54.42
UCB	0.34	15.49	−24	52.79
Static α=1	0.34	15.70	−23	52.86
Static α=0.5	0.34	16.10	−21	52.99
Static α=2	0.34	16.25	−20	53.05
Coverage-greedy	0.34	16.86	−17	53.25
Static α=0 (fixed depth)	0.34	16.91	−17	53.27
Pivotal	0.34	17.30	−15	53.40
Thompson sampling	0.34	18.43	−9	53.79
Tail-aware	0.34	20.63	+2	54.53
Learn-then-commit	0.34	21.91	+8	54.97
Info-greedy	0.34	36.68	+81	59.99

$\{0, 0.05, 0.15, 0.3, 0.5, 0.7, 0.9\}$, so that a fragment still missing after many reads is increasingly attributed to permanent absence rather than to low coverage — removes most of the penalty. On the baseline, against round-robin on the same 300 syntheses and read orders, UCB goes from +15.3% to −3.5% (95% CI −4.9 to −2.2%), Thompson sampling from +7.4% to +0.7% (−1.0 to +2.8%), coverage-greedy from +15.1% to +1.6% (−0.1 to +3.3%), pivotal from +49.3% to +7.4% and learn-then-commit from +44.6% to +12.7%. The mechanism is visible in the read shares: coverage-greedy's share of the near-dead container falls from 0.27 to 0.20 and UCB's from 0.27 to 0.17, against round-robin's 0.16. Information-greedy gets worse (+441% to +612%), which is consistent: a richer model has more uncertainty for a pure-information rule to chase. In the homogeneous control, where there is no dropout to detect, the two models agree to within 0.5% on every rule, so the second parameter costs nothing when it is not needed.

Two qualifications keep this from being a positive headline. First, the tail does not move: the paired q99 difference for UCB with the dropout model is −450 reads with a 95% interval of −1,147 to +150, so even the best rule is not resolved as better than round-robin at q99, and our statement that no rule lowered q99 survives the better model. Second, the size: the best allocation result in this paper is a 3.5% mean saving, against the 27–32% that verified stopping delivers with no model at all. The right reading of our negative result is therefore not "adaptive allocation does not work" but *"a container model that cannot express dropout makes adaptive allocation actively harmful, and fixing the model buys a few percent, not the bulk of the saving"*.

G. Failed containers and severity sweep

H. Failed containers and severity sweep

With one failed container (the fresh one, which returns only non-informative reads), 34% of retrievals succeeded under every policy, and success was identical for all policies on every synthesis and read order: whether the file can be recovered is set by which fragments the remaining five containers lost, i.e. by the archive's redundancy, not by the policy. On the 102 retrievals where all succeeded, adaptive allocation helped, because the failed container is easy to identify and stops being read: UCB used 24% (17–28%) fewer reads than round-robin, coverage-greedy 17% (8–23%), pivotal 15% (6–21%) and Thompson sampling 9% (2–15%); tail-aware was even (+2%, −8 to +14%), learn-then-commit 8% (2–14%) and information-greedy 81% (49–133%) costlier. With two failed containers no policy recovered the file; every retrieval read all 72,000 reads and reported failure. We read both results as boundaries of the *archive design*, not of the policies: with round-robin placement of $(32, 48)$ stripes over six containers, each failed container removes 8 of a stripe's 48 fragments, half of its redundancy of 16. After one failure a stripe tolerates only 8 further lost strands: 18 of the 30 syntheses were unrecoverable under every read order, 7 always recoverable, and 5 depended on the read order (consensus uses at most 24 reads per cluster); after two failures exactly K=32 fragments remain, so a single strand lost by synthesis, aging or PCR in any stripe makes the file unrecoverable. Surviving more failures requires more redundancy or a different placement; no read-allocation policy can compensate.

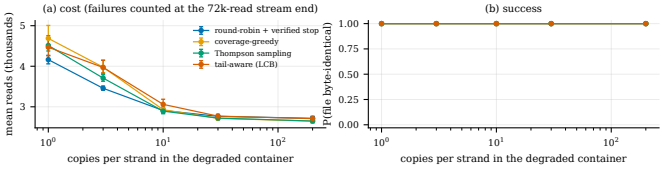


Fig. 3. Severity sweep: the near-dead container's copy number varied from 200 to 1 (300 retrievals per point and policy).

The severity sweep (Fig. 3) varies only the near-dead container's copies per strand. At 200 and 30 copies, coverage-greedy and Thompson sampling are 2–3% cheaper than round-robin (2.65k vs. 2.72k at 200 copies) and tail-aware is even; at 10 copies the policies are within $\pm 1\%$ except tail-aware ($+5\%$); at 3 and 1 copies all adaptive rules are costlier (at one copy: coverage-greedy 4.69k, Thompson 4.51k, tail-aware 4.47k vs. 4.16k). Success was 1.00 throughout. The point at 3 copies reproduces the baseline design with other seeds (round-robin 3.46k vs. 3.49k). Adaptive allocation therefore helps only while the model's single multiplier is a fair description of the worst container.

I. Tail metrics and the static family

No allocation rule lowered the 0.99-quantile, but the precision of that statement needs care. A q99 estimated from 300 retrievals is driven by its three most expensive observations, and those sit in a few synthesis replicates, so we report cluster-bootstrap intervals over syntheses (Table VIII). In the frozen design round-robin has q99 5.15k (95% CI 4.75–5.65k), Thompson sampling 6.45k (5.30–7.16k), tail-aware 7.15k (5.75–7.76k), UCB 8.15k (6.25–8.60k), coverage-greedy 9.05k (5.80–9.50k) and pivotal 9.60k (9.10–9.91k). The intervals of coverage-greedy and Thompson sampling overlap round-robin's, so for those two rules the direction is clear from the means but the q99 gap is not resolved by this design; pivotal, learn-then-commit and info-greedy are separated from round-robin at q99 as well as in the mean. The tail-aware rule, built to attack the bottleneck stripes, is worse than round-robin at q99 in every design we ran: the heuristic did not work. We therefore make only the claim the evidence supports — *no rule we tested lowered q99, and none of this machinery should be sold as tail-risk reduction* — and we report q95 beside q99 because it is the better-determined tail statistic at this sample size.

The static family deserves the same caution. Its members are $w_j \propto e_j^{\alpha}$, and the frozen protocol tested only $\alpha \in \{0, 0.5, 1, 2\}$, so "best static plan" means the best of those four, not the optimal fixed allocation. Revision 4 adds $\alpha \in \{0.25, 0.75, 1.25, 1.5, 3\}$ on the same replicates so that the family is sampled finely around its optimum, and, because the hindsight weights are measured on the evaluated syntheses and are therefore optimistic, a *calibration-estimated* static plan whose e_j come from the ten disjoint calibration replicates and which sees no evaluation data at all (Table V). A user who must reserve capacity gains nothing from stopping unless unused capacity is refunded; for that user the relevant comparison is between reservations, and a tuned static plan is the better choice. In the homogeneous control the static plans must reserve 2.85k against a round-robin consumed mean of 2.26k. With a failed container no static plan can be sized at all: it either reads the whole stream or fails, so its reservation is the full 72k reads.

J. Calibration of the posterior, and a negative forecast result

Two calibration questions must be kept apart: whether the posterior over θ_j is calibrated, and whether the completion forecast is. Neither is, and the first failure is specific.

Posterior intervals. On the ten selection replicates, 90% credible-interval coverage of θ^* rose with ρ_U from 0.45 (binomial) through 0.63, 0.73 and 0.80 to 0.87 at $\rho_U{=}0.1$, the edge of the pre-specified grid, which we kept and selected; on the 30 held-out replicates it gave 0.87 overall (0.84 at the earlier and 0.89 at the later read count), 1.00 for five containers and 0.20 for the near-dead one. That selection has two defects: it used prior sd $s{=}1$ while every policy run uses $s{=}0.5$, and the selected value sat at the grid boundary. Revision 4 repeats it with $s{=}0.5$ and an extended grid $\{0, 0.003, 0.01, 0.03, 0.1, 0.2, 0.3, 0.5\}$ (Table IX), and the outcome corrects the original selection in two ways. First, under the prior actually used the best value is $\rho_U{=}0.03$ (selection coverage 0.81, held out 0.83) rather than 0.1 (0.81 and 0.82), so the frozen default is not the best member of its own family, although the difference is within noise. Second, extending the grid does not rescue calibration: coverage flattens near 0.80 for $\rho_U \in [0.03, 0.3]$ and collapses to 0.60 at 0.5, so no value in this likelihood family reaches the 0.90 target. The reason is visible per container: at every ρ_U the five healthy containers are covered 0.92–1.00 of the time and the near-dead container 0.00–0.03 of the time. Widening a likelihood cannot fix a container whose behaviour the model cannot express, which is the same misspecification that makes allocation lose (Section VI-E). We therefore do not describe this controller as calibrated: its posterior is reliable for healthy containers and wrong for near-dead ones, which is precisely the case that matters.

Completion forecast (negative, secondary). The forecast carries no claim in this paper and no policy depends on it; we report it because it was pre-specified and because its failure is informative. On 1,127 checkpoints from 300 baseline retrievals with a budget of 3.75k reads (the median coverage-greedy cost), of which 52% finished within the budget, both forecasts were over-confident (mean forecast 0.86 against 0.52 observed at 30% of the budget). The posterior-simulation forecast beat the analytic one on every metric (ECE 0.29 vs. 0.34; Brier 0.30, 95% CI 0.27–0.34, vs. 0.34, 0.29–0.38; AUC 0.73, 0.69–0.78, vs. 0.69, 0.65–0.73), yet both Brier scores are worse than the 0.25 of a constant forecast at the observed base rate, so neither beats climatology. They rank retrievals usefully only late in a retrieval (AUC 0.82 at 85% of the budget). Futility stopping, which would rely on them, is disabled in every

TABLE VIII

TAIL METRICS WITH CLUSTER-BOOTSTRAP 95% INTERVALS OVER SYNTHESIS REPLICATES (BASELINE). EXTREME QUANTILES ESTIMATED FROM A FEW HUNDRED RETRIEVALS ARE IMPRECISE, AND THE INTERVALS OF MOST POLICIES OVERLAP; THE ORDERING OF THE MEANS DOES NOT CARRY OVER TO Q99. THOUSANDS OF READS.

Design	Allocation	$\bar{r}_{\text{all}}$	q95 [95% CI]	q99 [95% CI]
30 syntheses × 10 orders	Static α=1	3.34	4.45 [4.15, 4.55]	4.80 [4.65, 5.25]
	Round-robin + verified stop	3.49	4.56 [4.30, 4.75]	5.15 [4.75, 5.65]
	Static α=0 (fixed depth)	3.49	4.56 [4.30, 4.75]	5.15 [4.75, 5.65]
	Thompson sampling	3.75	5.21 [4.80, 6.10]	6.45 [5.30, 7.16]
	Tail-aware	3.94	5.65 [4.96, 6.19]	7.15 [5.75, 7.76]
	Coverage-greedy	4.02	5.80 [5.25, 8.05]	9.05 [5.80, 9.50]
	UCB	4.02	6.05 [5.20, 7.50]	8.15 [6.25, 8.60]
	Learn-then-commit	5.05	6.80 [6.35, 7.20]	7.65 [7.20, 8.10]
	Pivotal	5.21	8.85 [8.06, 9.25]	9.60 [9.10, 9.90]
	Info-greedy	18.87	31.44 [27.61, 34.22]	37.45 [33.70, 37.51]

TABLE IX

RE-RUN OF THE ρ_U SELECTION WITH THE PRIOR ACTUALLY USED IN THE POLICY RUNS (s=0.5) AND AN EXTENDED GRID: COVERAGE OF THE 90% CREDIBLE INTERVAL OF θ_j^*, OVERALL AND PER CONTAINER. NO VALUE IN THE GRID REACHES 0.90 OVERALL, AND NO VALUE COVERS THE NEAR-DEAD CONTAINER.

Split	ρ_U	All	fresh	aged 1 hl	aged 2 hl, 10 cop.	30 cyc.	45 cyc., 20 cop.	near-dead
select (reps 1000-1009)	0.0	0.43	0.80	0.60	0.00	0.75	0.45	0.00
	0.003	0.64	1.00	0.85	0.25	0.95	0.80	0.00
	0.01	0.74	1.00	0.90	0.55	1.00	1.00	0.00
	0.03	0.81	1.00	0.95	0.90	1.00	1.00	0.00
	0.1	0.81	0.95	0.90	1.00	1.00	1.00	0.00
	0.2	0.80	0.90	0.90	1.00	1.00	1.00	0.00
	0.3	0.80	0.90	0.90	1.00	1.00	1.00	0.00
	0.5	0.60	0.50	0.60	1.00	0.50	1.00	0.00
validate (reps 0-29)	0.0	0.54	0.78	0.88	0.23	0.78	0.52	0.02
	0.003	0.67	0.97	1.00	0.37	0.97	0.68	0.03
	0.01	0.76	1.00	1.00	0.62	1.00	0.93	0.03
	0.03	0.83	1.00	1.00	0.95	1.00	1.00	0.03
	0.1	0.82	0.92	1.00	1.00	0.97	1.00	0.03
	0.2	0.80	0.87	0.97	1.00	0.93	1.00	0.03
	0.3	0.80	0.87	0.97	1.00	0.93	1.00	0.03
	0.5	0.59	0.52	0.55	1.00	0.48	0.98	0.03

run, and we make no claim that these numbers are usable probabilities.

K. Secondary evidence: real read streams

Table X gives the leak-free replay with the generic prior. On the Twist pool, round-robin with verified stopping used 58.6k reads on average against the 65.7k budget of the hindsight-tuned static plan (-11%), and the model-based rules used 56.6–57.2k, 2–3% fewer than round-robin; coverage-greedy was 9.0k (95% CI 8.5–19.3k) below the hindsight static budget. Information-greedy reduced to coverage-greedy after the first probes (identical results), and the leak-free calibrated belief changed coverage-greedy by -0.2%. As on the digital twin, no rule lowered q99 materially: 65.6–67.2k for the model-based rules against the 65.7k static budget. Stopping supplies most of the gain (about 7.1k of 9.0k reads). On the Genscript pool, whose PCR-30-cycle container is near-dead, verified stopping with round-robin used 637k reads against 852k for fixed depth (-25%), but 1% more than the 631k of the hindsight-tuned static plan, which knows in advance to give that container 2.7% of its reads (paired difference $+6.0$k, 95% CI -57.4 to $+15.1$k). Every model-based rule with the generic prior was 0–3% costlier than round-robin (UCB 637k, coverage-greedy and pivotal 639k, tail-aware 644k, learn-then-commit 651k, Thompson sampling 654k), and, as on the digital twin, each gave the near-dead container more reads than round-robin did (19–20% against 17%). The exception is the leak-free calibrated belief, fitted on calibration reads of the same containers that are disjoint from the evaluation reads: coverage-greedy with it gave the near-dead container 3.4% of its reads and used 533k (-16% vs. round-robin; -97.9k, 95% CI -161.6 to -90.0k, vs. the hindsight static budget). Allocation therefore pays when the weak container is identified *before* retrieval, not when the controller must learn it online. q99 was 820–853k for the generic-prior rules and 676k with the calibrated belief, against the 631k static budget. The information-greedy run on this pool did not complete in time (Appendix B).

L. Supporting channel result (real nanopore reads)

On two independent nanopore flowcells of DNAformer reads [41], [50], a strand-specific error propensity persists across flowcells (r=0.57; null with matched contexts 0.002), and a hierarchical beta-binomial error model fitted on one flowcell predicts consensus success on held-out strands of the other with mean absolute error 0.083, against 0.181 for an i.i.d. model. This motivates over-dispersed models but does not enter the controller.

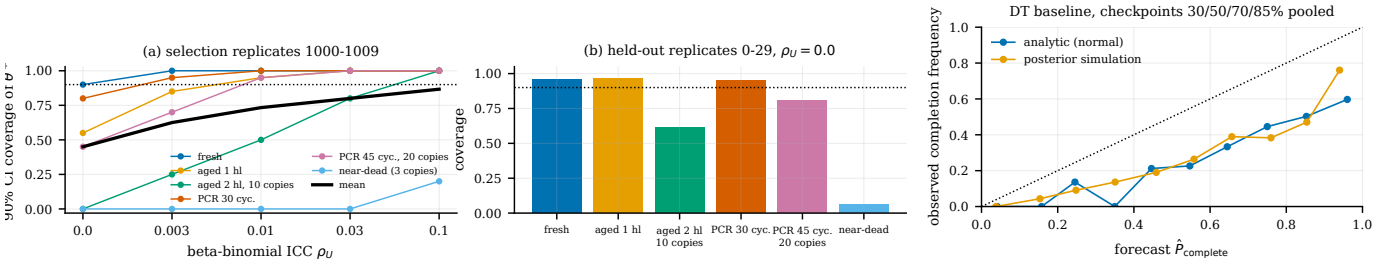


Fig. 4. Calibration on the digital twin. Left: 90% credible-interval coverage of θ^* per container on the selection replicates for each ρ_U, and on held-out replicates at the selected ρ_U. Right: reliability of the completion forecast (analytic vs. posterior simulation).

TABLE X
SECONDARY EVIDENCE: SEMI-EMPIRICAL REPLAY OF REAL ILLUMINA READ STREAMS (ENA PRJEB65931), GENERIC PRIOR, NO EVALUATION-SPLIT STATISTICS. MEAN READS (THOUSANDS) AND Q99; Δ: MEAN MINUS THE HINDSIGHT-TUNED STATIC BUDGET (PAIRED BOOTSTRAP 95% CI). DECODABILITY IS AN ORACLE EVENT HERE (CONSENSUS EQUALS THE DESIGNED STRAND), SO THIS IS *not* EXACT RETRIEVAL.

Pool	Policy	Mean	q99	Δ vs. static [95% CI]
Twist	Static, uniform (hindsight budget)	68.1	68.1	
	Static, hindsight-tuned	65.7	65.7	
	Round-robin + verified stop	58.6	68.4	-7.1 [-17.4, -6.5]
	Coverage-greedy	56.7	66.0	-9.0 [-19.3, -8.5]
	Pivotal	56.6	67.2	-9.1 [-19.4, -8.6]
	Thompson sampling	56.9	66.0	-8.9 [-19.1, -8.4]
	UCB	56.7	66.6	-9.0 [-19.2, -8.5]
	Learn-then-commit	57.2	66.6	-8.5 [-18.8, -8.0]
	Info-greedy	56.7	66.0	-9.0 [-19.3, -8.5]
	Tail-aware (LCB)	56.7	65.6	-9.0 [-19.3, -8.5]
	Coverage-greedy, leak-free calibrated	56.6	67.6	-9.1 [-19.4, -8.6]
Genscript	Static, uniform (hindsight budget)	852	852	
	Static, hindsight-tuned	631	631	
	Round-robin + verified stop	637	853	+6.0 [-57.4, +15.1]
	Coverage-greedy	639	820	+8.2 [-54.9, +17.2]
	Pivotal	639	820	+8.2 [-54.9, +17.2]
	Thompson sampling	654	831	+23.4 [-39.4, +31.9]
	UCB	637	830	+6.0 [-57.8, +14.9]
	Learn-then-commit	651	843	+19.7 [-42.8, +29.2]
	Info-greedy	not completed (Appendix B)		
	Tail-aware (LCB)	644	827	+12.9 [-51.0, +21.6]
	Coverage-greedy, leak-free calibrated	533	676	-97.9 [-161.6, -90.0]

VII. DISCUSSION

What helps, and whom. The robust gain in our experiments is verified stopping: reading until every stripe, its CRC-32 and the file hash verify, and then stopping. It needs no model of the containers, never returned a wrong file, and converts the capacity a static plan must reserve into a bill 27–32% smaller for a user who pays per read. It does *not* reduce the capacity a user must reserve, because the q99 is unchanged. Choosing *which* container to read next is a second-order effect in every configuration we tested: at best 3.5% (UCB with a dropout-aware container model, or 4.2% for a hindsight static plan that already knows the answer), and strongly negative when the container model cannot express what is wrong with a container. If one sentence has to carry the paper's practical content, it is that the decision worth engineering is *when to stop*, and the decision worth engineering next is the *container model*, not the allocation rule on top of it.

Calibration versus cost, and model versus likelihood. The likelihood that gave the best-calibrated intervals (ρ_U=0.1) also gave the costlier allocations, and the binomial likelihood, whose intervals covered only 54%, gave cheaper ones. Calibrating the uncertainty of a misspecified model does not make its decisions better. Our revision-4 experiments go one step further and say which part was at fault: re-selecting ρ_U with the prior actually used and on a wider grid still cannot cover the near-dead container at any value, while adding a dropout parameter to the container model fixes most of the cost penalty. The lesson we draw is about priorities — repair the container model before tuning the posterior over it — and we therefore present the controller as online-adaptive, not self-calibrating, and do not offer its forecasts as probabilities.

Digital twin versus replay. The two settings agree that stopping is the main saving and that no rule lowers q99; on allocation with the generic prior they agree where the conditions match. On the Twist pool, whose containers are runs of one synthesis batch with no near-dead member (the mild-heterogeneity end of the sweep), allocation saved 2–3%; on the Genscript pool, which has a near-dead container, it cost 0–3% and over-read that container, as on the digital twin. Only prior knowledge of the weak container, through calibration reads, turned allocation into a large saving.

Practical recommendation. The box in Fig. 5 summarises what our results support for archives like ours.

What we would test with wet-lab containers. With physically separate containers we would test (X) whether real containers show the near-dead, dropout-dominated behaviour that made adaptive allocation lose here, by synthesising one design in several independent batches, aging and amplifying them differently, and sequencing each container to saturation to measure its fraction of absent strands; (Y) whether verified stopping keeps its saving and its safety on real sequencing runs with our own inner code, by streaming reads from a nanopore or a split Illumina run through the same decoder and counting wrong outputs and inner-code false accepts against the known file; and (Z) what real container failures look like and how often they occur, by tracking a set of containers over accelerated aging, which would calibrate the failure prior and the redundancy needed per container. The decisive experiment, which we could not run, is the full chain on physical material: encode a known file with these codes, synthesise it, split it into

Practical recommendation (multi-container retrieval).

1) Read the containers in turn and **stop on verification** (every stripe against its CRC-32, the file against its hash). This is the robust saving, 27–32% of a reserved budget, and it needs no model. Never return an unverified file.
2) If you **reserve** capacity rather than pay per read, expect no saving at all from any of this: budget the q99 of round-robin, and prefer a tuned static allocation.
3) **Switch a container off** once it is clearly failed (no verified fragment after a probe). Posterior rules cut the cost of the retrievals that still succeed by up to 24%, but they do not make recovery more likely.
4) Do **not** put a model-based allocation rule in front of a container model that cannot represent dropout: every such rule we tried read more than round-robin, by 7–441%. With efficiency *and* dropout, the best rule gained 3.5%; decide whether that is worth the complexity.
5) If calibration reads of the same containers are available, spend them: a prior fitted on held-out reads identified the weak container in advance and saved 16% on one real-read pool, which is the largest allocation effect we measured.
6) **Size redundancy for the failures you must survive.** With $(32, 48)$ over six containers, two lost containers leave exactly K fragments per stripe and no margin; and a noisier synthesis model made the file unrecoverable in our design whatever the policy.

Fig. 5. Practical recommendation derived from Section VI.

independently synthesised and separately handled containers, sequence each one, feed the FASTQ reads into this decoder, and compare the returned bytes with the original. Until that is done, the replay in Section VI-K remains semi-empirical — real read streams, oracle decodability — and the end-to-end evidence remains simulated.

VIII. Limitations and Threats to Validity

Table XI lists the main threats. *External validity*: every result is computational. The digital twin uses DT4DDS's default parameters and one archive geometry (1,200 strands, $(32, 48)$, six containers), and its failure mode is simulated (a failed container returns non-informative reads). The replay uses real reads, but decodability is an oracle event and the "containers" are runs of one synthesis batch. *Internal validity*: ρ_U was selected with prior sd $s{=}1$ and used with $s{=}0.5$; the selected value lies at the edge of the pre-specified grid; the photolithographic scenario uses a reduced design; and one code fix (static allocator) was made after freezing (Appendix B). The static baselines use efficiencies measured in hindsight on the evaluated syntheses, which favours them. *Construct validity*: read counts are a proxy for cost; we do not model run set-up costs, which favour fewer, larger batches. *Statistical*: wrong-output and false-accept bounds come from zero events and scale with the number of trials; the comparisons are paired but within one archive design. We prove no optimality or regret result.

TABLE XI
Assumptions, limitations and their consequence. Items marked † were raised against an earlier revision and are now measured rather than only acknowledged; the measurement is cited.

Limitation	Consequence
Computational only (digital twin + replay)	no wet-lab confirmation; DT4DDS fidelity bounds external validity; see Section VII for what we would test
Single efficiency multiplier θ_j in the frozen model	cannot represent dropout; adaptive allocators over-read near-dead containers. † A two-parameter model (efficiency and dropout) removes most of the penalty (Section VI-F)
ρ_U at the edge of its grid; selected with $s{=}1$ while runs use $s{=}0.5$	† re-selected with $s{=}0.5$ on an extended grid: no value reaches 0.90 coverage and the near-dead container is never covered (Table IX)
Calibration-optimal vs. cost-optimal ρ_U differ	a better-calibrated posterior allocated worse; the ablation separates likelihood, prior and model (Table **??**)
Gains are in the mean, not the tail	† no rule lowered q99; tail intervals are wide and reported (Table VIII)
Failures simulated	real failure modes and rates unknown; † partial loss, low yield, contamination and common-cause degradation are now tested (Table **??**), but their rates are still assumed
One archive geometry in the primary design	† six further geometries tested (Table **??**); conclusions outside those remain untested
Consensus cap of 24 reads per cluster	† caps of 12 and 48 tested (Table **??**); decoder configuration does affect the recovery boundaries
Replay is semi-empirical	oracle decodability, so it is *not* end-to-end exact retrieval; containers are runs of one synthesis batch, not independent syntheses
Genscript pool: 472 of 12,472 designs excluded	4.23% of mapped evaluation reads discarded; identical material in every condition
Read count is not money	† per-batch and per-container charges change the ranking (Table **??**); instrument time, library preparation and minimum run sizes are not modelled
Container choice every 50 reads	an allocation policy over independent read streams, not a demonstrated physical sequencing controller (Section IV-C)
Safety bounds assume independence per retrieval	† retrievals share syntheses; the conservative per-synthesis bound is 10^{-2}, not 10^{-4} (Table IV)
Nanopore correlation evidence: two flowcells	preliminary supporting evidence only; does not validate the container model
Information-greedy on the Genscript replay did not complete	reported as censored, not as a result (Appendix B)
No theory (regret, optimality)	empirical claims only, about the configurations tested

IX. Reproducibility and Availability

Artifact. Code, configuration, every result file and the scripts that build every table and figure are in the public repository https://github.com/SaiMouli3/Dev (directory `d3ra-dna`), at the commit recorded in `research_paper/final/COMMIT.txt` of that revision. The environment is Python 3.13.18 with NumPy 2.5.3, SciPy 1.18.1, pandas 3.0.5 and DT4DDS 1.1.0 (pinned in `requirements.txt`); the public data are ENA PRJEB65931 [45] (Illumina) and the DNAformer nanopore reads [**?**], [50], both fetched by the scripts in `experiments/`. `research_paper/audit/` holds the claim-to-number recomputation and its output.

What reproduces what. The digital-twin experiments are driven by `experiments/dt_experiments.py` (sub-commands `gen`, `run`, `runx`, `static`, `calib`, `calib_eval`, `forecast`, `posthoc_rho0`), the replay by `experiments/paper_experiments.py` (`replay_v3`, `replay_calcost`); statistics by `dt_analysis.py` and `r4_analysis.py`; tables and

figures by `make_tables.py`, `make_r4_tables.py` and `make_dt_figures.py`, which read only committed result files, so no number in any table or figure is typed in by hand. Table XII maps each claim to its command and result file, and `dt_protocol.yaml` and `dt_protocol_rev4.yaml` record the frozen protocols and every deviation.

Determinism, and the scope of our re-run checks. Every random draw is seeded by CRC-32 of a string naming the scenario, replicate, container, read order and policy, so results do not depend on process scheduling or worker count. We verify this at three scales and state what each covers. (i) *Field level*: 24 randomly sampled baseline retrievals, re-executed in a fresh single-process run in reverse order, reproduced every recorded field (status, reads, per-container reads, rounds, decode attempts, false accepts, verification rounds) exactly. (ii) *Condition level*: the Genscript replay conditions, re-run in separate parallel processes after a code fix, reproduced the values of the original sequential run exactly. (iii) *Scenario level*: the primary baseline scenario and its static family were regenerated from the final commit and every summary number in this paper recomputed from those files. We did *not* re-execute the whole paper: the severity sweep, the failure scenarios, the alternative geometries, the photolithographic scenario and the two replays are reproduced from their committed result files and seeds. A full re-execution takes several CPU-days, dominated by DT4DDS synthesis and by the information-greedy policy, and the artifact contains the commands to do it.

X. Conclusion

On a digital twin with independently synthesised containers, verified stopping retrieved the file without a single wrong output and cut the mean reads per retrieval by 27% against the best static reservation; adaptive container selection added nothing when a container was near-dead and helped (by up to 24%) only after an outright failure or under mild heterogeneity. No rule lowered the 0.99-quantile, and the controller's forecasts are over-confident. The results are computational, for one archive design and one simulator; their value is a reproducible, leak-free baseline and a clear account of where online adaptation fails. A container model with dropout, wet-lab containers and a real failure distribution are the next steps.

Appendix A
Upper bound with evaluation leakage

Our first analysis of the real read streams (Table XIII) used a "calibrated" belief whose per-strand consensus curves were pooled over reads that included the evaluation split. The controller therefore had information about the very reads it was evaluated on, and the static plans were tuned on them too. We keep the table only as an *upper bound with evaluation leakage*; no claim in the main text uses it. In the leak-free re-analysis (Table X) the primary belief is the generic prior, and the optional calibrated belief uses only the first 300k reads of each run while evaluation uses the next 900k. On Twist, the leaky analysis put coverage-greedy at 55.7k reads and the oracle-tuned static plan at 63.1k; the leak-free analysis gives 56.7k and 65.7k. The leak therefore flattered the adaptive policy by about 1k reads and the static baseline by about 2.6k, and the leak-free gap is larger, not smaller.

Appendix B
Protocol deviations and post-hoc analyses

Deviations. The frozen protocol and every change made after freezing are in `dt_protocol.yaml`. (i) The planned CustomArray scenario was replaced, before any policy run, by DT4DDS's photolithographic synthesis model, because the CustomArray settings carry a 117-nt position-bias table that cannot be applied to 136-nt strands. (ii) We suspected from commit timestamps that the first Twist replay had run with $\rho_U{=}0$ and re-ran it; a direct test showed that it had used the frozen $\rho_U{=}0.1$, and the re-run reproduced it exactly, so nothing changed. (iii) One cached homogeneous replicate was truncated by an interrupted job; it was regenerated from its seed and the scenario re-run in full. (iv) The photolithographic scenario was run with 10 syntheses $\times$ 3 read orders for compute reasons; the first 50 retrievals of the full run were discarded unseen. (v) The static allocator crashed when only zero-weight (failed) containers remained; it now reads them out, which can only affect failed retrievals of the failure scenarios. The ρ_U rule was applied as frozen even though the selected value lies at the edge of the grid. (vi) The sequential Genscript replay finished information-greedy after about six hours but then crashed while computing its paired difference, because some trials never completed (infinite cost); nothing was saved. We fixed the bug, ran the other Genscript conditions in parallel processes with the same per-condition seeds (they reproduced every value logged by the sequential run), and re-started information-greedy; it had not finished when this version was built, so Table X has no information-greedy row for Genscript. What we know about that condition is reportable and we report only that: in the sequential run its 200 trials finished, and more than 1% of them never recovered the file within the available reads, which is why the 0.99 budget was infinite and the paired statistic overflowed. We treat the condition as *censored* — excluded from every average and comparison, marked as not completed in Table X — and we make no claim anywhere that every policy completed every real-read condition. (vii) Revision 4 adds the pre-registered secondary analyses of `dt_protocol_rev4.yaml`: the dropout-aware container model, the $2{\times}2{\times}2$ ablation, the ρ_U re-selection at $s{=}0.5$ on an extended grid, prior sensitivity, five further failure modes, six further archive geometries, consensus caps of 12 and 48, finer and calibration-estimated static families, an extended baseline replicate set, the full-design photolithographic run, the replay calibration-cost experiment and the cost model. The frozen revision-3 settings remain the primary configuration; every revision-4 result is secondary and is reported whatever its direction.

Post-hoc analyses (not pre-specified). With the binomial likelihood ($\rho_U{=}0$), on the same 300 baseline retrievals, coverage-greedy used 3.34k reads (-4.4% vs. round-robin,

TABLE XII

CLAIM-TO-EVIDENCE SUMMARY (FULL TABLE: `EVIDENCE/EVIDENCE_TRACEABILITY.CSV`; EVERY NUMBER RECOMPUTED FROM THESE FILES BY `AUDIT/RECOMPUTE_CLAIMS.PY` AND `R4_ANALYSIS.PY`). DT: DT4DDS DIGITAL TWIN, FULL DECODE CHAIN, NO ORACLE. REPLAY: REAL READ STREAMS, ORACLE DECODABILITY. NO RESULT IN THIS PAPER IS WET-LAB EVIDENCE.

Claim	Command (`dt_experiments.py` unless noted)	Result file (`results/`)	Evidence
No wrong output, no false accept; cluster-aware bounds (Table IV)	all DT runs	`dt_*.csv` → `r4_safety.csv`	DT
Stopping converts a 4.80–5.15k reservation into a 3.49k bill (Table V)	`run`/`static` baseline	`dt_baseline{,_static}.csv` → `r4_actual_vs_reserved.csv`	DT
Hindsight static with stopping beats round-robin by 4.2%	`static` baseline (+`_r4fine`, `_r4cal`)	`dt_baseline_static{,_r4fine,_r4cal}.csv`	DT
Model-based rules costlier than round-robin under the frozen model	`run` baseline	`dt_summary.csv`	DT, negative
Cause is the one-parameter model: adding dropout removes most of the penalty (Table **??**)	`runx SCEN _r4drop`	`dt_*_r4drop.csv` → `r4_ablation.csv`	DT
Likelihood, prior and model separated by ablation	`runx baseline _r4abl_*`	`dt_*_r4abl_*.csv`	DT
After an outright failure: -24% on successes, -3% per attempt, success unchanged (Table VII)	`run fail1`	`dt_fail1{,_static}.csv`	DT, simulated failure
Crossover with severity of the worst container	`run sweep_cov*`	`dt_sweep_cov*.csv`	DT
No rule lowers q99; wide tail intervals (Table VIII)	all DT runs, `_r4ext`	`r4_tails.csv`	DT, negative
ρ_U re-selected at s=0.5 on an extended grid; near-dead never covered (Table IX)	`calib`, `calib_eval` (s=0.5)	`dt_calibration_*_r4sd05.csv`	DT, negative
Forecasts over-confident, worse than climatology	`forecast baseline`	`dt_forecast_baseline.csv`	DT, negative
Prior sensitivity (failure rate, dropout, shape, grid, s, φ)	`runx baseline _r4prior_*`	`r4_prior_sensitivity.csv`	DT
Failure modes: partial loss, low yield, contamination, common cause (Table **??**)	`runx {partial,lowyield,contam,common}*`	`r4_failure_modes.csv`	DT, simulated
Six further archive geometries (Table **??**)	`D3RA_GEOM=... runx baseline _r4`	`r4_geometries.csv`	DT
Consensus cap 12/24/48 (Table **??**)	`runx ... _r4cap{12,48}`	`r4_consensus_cap.csv`	DT
Homogeneous control; photolithographic boundary	`run homog`, `run photolitho`	`dt_{homog,photolitho}{,_static}.csv`	DT
Cost model with per-batch and per-container charges (Table **??**)	from committed per-retrieval rows	`r4_cost_model.csv`	DT, assumed prices
Leak-free real-read replay, both pools	`paper_experiments.py replay_v3`	`replay_v3{,_genscript}.csv`	Replay
Calibrated prior pays for itself after $\sim$1 retrieval (Table **??**)	`paper_experiments.py replay_calcost`	`r4_replay_calcost.csv`	Replay
Leaky first analysis (upper bound only)	`real_data.py policies`	`realdata_*policies*_v2.csv`	superseded
Correlated nanopore errors (MAE 0.083 vs. 0.181)	`q3_correlated_errors.py`	`q3_summary.json`	real nanopore reads
Regret bounds; wet-lab containers; real failure rates; independently synthesised physical containers	–	–	not attempted

TABLE XIII

UPPER BOUND WITH EVALUATION LEAKAGE (SUPERSEDED FIRST ANALYSIS, KEPT FOR TRANSPARENCY). REPLAY OF REAL READ STREAMS WITH "CALIBRATED" BELIEFS WHOSE CONSENSUS CURVES WERE POOLED FROM *evaluation*-SPLIT READS (APPENDIX A); THOUSANDS OF READS AT 0.99. THESE NUMBERS ARE NOT USED FOR ANY CLAIM IN THE MAIN TEXT. STATIC PLANS PAY THEIR BUDGET ON EVERY RETRIEVAL; FOR ADAPTIVE POLICIES THE 0.99-QUANTILE Q99 IS SHOWN SEPARATELY. Δ: ADAPTIVE (PIVOTAL) MINUS ORACLE-TUNED STATIC, PAIRED (95% CI).

Pool	Placement	Fixed depth	Oracle-tuned static	Adaptive stop, round-robin	Adaptive, cov.-greedy	Adaptive, pivotal	q99 of pivotal	Δ (95% CI)
Twist	round-robin (6)	67.9	63.1	57.8	55.7	55.8	63.2	-7.3 (-12.1, -7.0)
Twist	reliability-weighted	65.4	61.9	56.0	54.2	54.1	61.8	-7.9 (-13.3, -7.6)
Twist	tiered (4)	61.9	53.1	51.3	46.4	46.2	53.0	-6.9 (-19.1, -6.2)
Twist	single best	53.9	53.9	44.8	44.8	44.8	54.0	-9.1 (-11.0, -6.7)
Genscript	round-robin (6)	810.4	617.9	639.9	532.1	531.9	694.2	-86.0 (-127.1, -78.8)
Genscript	single best†	279.4	279.4	219.0	219.0	219.0	279.6	-60.4 (-70.5, -54.2)

200 read permutations, except †100. All policies completed in every trial. The single-best container is chosen with the calibrated belief, which contains the evaluation-split leak of Appendix A. Superseded first-run Genscript numbers (computed before the per-strand consensus cache was regenerated) are kept in `results/realdata_genscript_policies_{roundrobin,part2}.csv`.

against $+15.1\%$ at ρ_U=0.1), Thompson sampling 3.37k (-3.5%, against $+7.4\%$), tail-aware 3.37k (-3.5%, against $+12.9\%$) and pivotal 3.64k ($+4.4\%$, against $+49.3\%$); q99 was 4.95–5.70k, still not below round-robin's 5.15k. With one failed container the binomial likelihood helped tail-aware (-25% vs. $+2\%$) and Thompson sampling (-20% vs. -9%) and hurt coverage-greedy and pivotal slightly (-15% vs. -17%; -8% vs. -15%). Success and safety were unchanged (no wrong output). On the Twist replay, the binomial likelihood made coverage-greedy and Thompson sampling costlier (57.6k each, against 56.7k and 56.9k at ρ_U=0.1), the opposite of the digital-twin result: the two settings disagree on which likelihood allocates better.

ACKNOWLEDGMENT

We thank the authors of the public datasets and of DT4DDS (ENA PRJEB65931; DNAformer; Srinivasavaradhan *et al.*) for releasing reads, designs and software.

REFERENCES

[1] G. M. Church, Y. Gao, and S. Kosuri, "Next-generation digital information storage in DNA," *Science*, vol. 337, no. 6102, p. 1628, 2012.

[2] N. Goldman, P. Bertone, S. Chen, C. Dessimoz, E. M. LeProust, B. Sipos, and E. Birney, "Towards practical, high-capacity, low-maintenance information storage in synthesized DNA," *Nature*, vol. 494, no. 7435, pp. 77–80, 2013.

[3] R. N. Grass, R. Heckel, M. Puddu, D. Paunescu, and W. J. Stark, "Robust chemical preservation of digital information on DNA in silica with error-correcting codes," *Angewandte Chemie International Edition*, vol. 54, no. 8, pp. 2552–2555, 2015.

[4] Y. Erlich and D. Zielinski, "DNA fountain enables a robust and efficient storage architecture," *Science*, vol. 355, no. 6328, pp. 950–954, 2017.

[5] L. Organick, S. D. Ang, Y.-J. Chen, R. Lopez, S. Yekhanin, K. Makarychev, M. Z. Racz, G. Kamath, P. Gopalan, B. Nguyen, C. N. Takahashi, S. Newman, H.-Y. Parker, C. Rashtchian, K. Stewart, G. Gupta, R. Carlson, J. Mulligan, D. Carmean, G. Seelig, L. Ceze, and K. Strauss, "Random access in large-scale DNA data storage," *Nature Biotechnology*, vol. 36, no. 3, pp. 242–248, 2018.

[6] R. Heckel, G. Mikutis, and R. N. Grass, "A characterization of the DNA data storage channel," *Scientific Reports*, vol. 9, no. 1, p. 9663, 2019.

[7] A. L. Gimpel, W. J. Stark, R. Heckel, and R. N. Grass, "A digital twin for DNA data storage based on comprehensive quantification of errors and biases," *Nature Communications*, vol. 14, no. 1, p. 6026, 2023.

[8] D. Bar-Lev, O. Sabary, R. Gabrys, and E. Yaakobi, "Cover your bases: How to minimize the sequencing coverage in DNA storage systems," *IEEE Transactions on Information Theory*, vol. 71, no. 1, pp. 192–218, 2025.

[9] A. Gruica, D. Bar-Lev, A. Ravagnani, and E. Yaakobi, "A combinatorial perspective on random access efficiency for DNA storage," in *Proc. IEEE International Symposium on Information Theory (ISIT)*. IEEE, 2024, pp. 675–680.

[10] J. Bornholt, R. Lopez, D. M. Carmean, L. Ceze, G. Seelig, and K. Strauss, "A DNA-based archival storage system," in *Proceedings of the 21st International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*. ACM, 2016, pp. 637–649.

[11] X. Kong, O. Elishco, C. Wang, and T. M. Duman, "The coverage depth problem in distributed DNA data storage," arXiv:2610.02931 [cs.IT], 2026, preprint.

[12] A. Levy, R. Con, E. Yaakobi, and H. M. Kiah, "Expected recovery time in DNA-based distributed storage systems," in *Proc. IEEE International Symposium on Information Theory (ISIT)*, Jun. 2026, pp. 1–6.

[13] R. Lopez, Y.-J. Chen, S. Dumas Ang, S. Yekhanin, K. Makarychev, M. Z. Racz, G. Seelig, K. Strauss, and L. Ceze, "DNA assembly for nanopore data storage readout," *Nature Communications*, vol. 10, no. 1, p. 2933, 2019.

[14] W. R. Thompson, "On the likelihood that one unknown probability exceeds another in view of the evidence of two samples," *Biometrika*, vol. 25, no. 3-4, pp. 285–294, 1933.

[15] P. Auer, N. Cesa-Bianchi, and P. Fischer, "Finite-time analysis of the multiarmed bandit problem," *Machine Learning*, vol. 47, no. 2–3, pp. 235–256, 2002.

[16] A. Garivier, T. Lattimore, and E. Kaufmann, "On explore-then-commit strategies," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 29, 2016.

[17] K. Chaloner and I. Verdinelli, "Bayesian experimental design: A review," *Statistical Science*, vol. 10, no. 3, pp. 273–304, 1995.

[18] P. L. Antkowiak, J. Lietard, M. Z. Darestani, M. M. Somoza, W. J. Stark, R. Heckel, and R. N. Grass, "Low cost DNA data storage using photolithographic synthesis and advanced information reconstruction and error correction," *Nature Communications*, vol. 11, no. 1, p. 5345, 2020.

[19] W. H. Press, J. A. Hawkins, S. K. Jones, J. M. Schaub, and I. J. Finkelstein, "HEDGES error-correcting code for DNA storage corrects indels and allows sequence constraints," *Proceedings of the National Academy of Sciences*, vol. 117, no. 31, pp. 18 489–18 496, 2020.

[20] Y.-J. Chen, C. N. Takahashi, L. Organick, C. Bee, S. D. Ang, P. Weiss, B. Peck, G. Seelig, L. Ceze, and K. Strauss, "Quantifying molecular bias in DNA data storage," *Nature Communications*, vol. 11, no. 1, p. 3264, 2020.

[21] I. Shomorony and R. Heckel, "DNA-based storage: Models and fundamental limits," *IEEE Transactions on Information Theory*, vol. 67, no. 6, pp. 3675–3689, 2021.

[22] N. Weinberger and N. Merhav, "The DNA storage channel: Capacity and error probability bounds," *IEEE Transactions on Information Theory*, vol. 68, no. 9, pp. 5657–5700, 2022.

[23] A. Lenz, P. H. Siegel, A. Wachter-Zeh, and E. Yaakobi, "Coding over sets for DNA storage," *IEEE Transactions on Information Theory*, vol. 66, no. 4, pp. 2331–2351, 2020.

[24] R. Cao and X. Chen, "Optimizing sequencing coverage depth in DNA storage: Insights from DNA storage data," in *Proc. IEEE International Symposium on Information Theory (ISIT)*. IEEE, 2025.

[25] M. Bertuzzo, A. Ravagnani, and E. Yaakobi, "The DNA coverage depth problem: Duality, weight distributions, and applications," arXiv:2603.06489 [cs.IT], 2026, preprint.

[26] S. Kas Hanna, "On the reliability of information retrieval from MDS coded data in DNA storage," *IEEE Transactions on Molecular, Biological, and Multi-Scale Communications*, vol. 12, pp. 585–598, 2026.

[27] M. Loose, S. Malla, and M. Stout, "Real-time selective sequencing using nanopore technology," *Nature Methods*, vol. 13, no. 9, pp. 751–754, 2016.

[28] A. Payne, N. Holmes, T. Clarke, R. Munro, B. J. Debebe, and M. Loose, "Readfish enables targeted nanopore sequencing of gigabase-sized genomes," *Nature Biotechnology*, vol. 39, no. 4, pp. 442–450, 2021.

[29] L. Weilguny, N. De Maio, R. Munro, C. Manser, E. Birney, M. Loose, and N. Goldman, "Dynamic, adaptive sampling during nanopore sequencing using Bayesian experimental design," *Nature Biotechnology*, vol. 41, no. 7, pp. 1018–1025, 2023.

[30] D. J. Russo, B. Van Roy, A. Kazerouni, I. Osband, and Z. Wen, "A tutorial on Thompson sampling," *Foundations and Trends in Machine Learning*, vol. 11, no. 1, pp. 1–96, 2018.

[31] T. Lattimore and C. Szepesvári, *Bandit Algorithms*. Cambridge University Press, 2020.

[32] S. Weil, S. Brandt, E. Miller, and C. Maltzahn, "CRUSH: Controlled, scalable, decentralized placement of replicated data," in *Proceedings of the 2006 ACM/IEEE Conference on Supercomputing (SC'06)*. IEEE, 2006, p. 31.

[33] A. Cidon, S. Rumble, R. Stutsman, S. Katti, J. Ousterhout, and M. Rosenblum, "Copysets: Reducing the frequency of data loss in cloud storage," in *2013 USENIX Annual Technical Conference (USENIX ATC '13)*. San Jose, CA, USA: USENIX Association, 2013, pp. 37–48. [Online]. Available: https://www.usenix.org/conference/atc13/technical-sessions/presentation/cidon

[34] D. Ford, F. Labelle, F. I. Popovici, M. Stokely, V.-A. Truong, L. Barroso, C. Grimes, and S. Quinlan, "Availability in globally distributed storage systems," in *9th USENIX Symposium on Operating Systems Design and Implementation (OSDI '10)*. Vancouver, BC, Canada: USENIX Association, 2010. [Online]. Available: https://www.usenix.org/conference/osdi10/availability-globally-distributed-storage-systems

[35] S. Nath, H. Yu, P. B. Gibbons, and S. Seshan, "Subtleties in tolerating correlated failures in wide-area storage systems," in *3rd USENIX Symposium on Networked Systems Design and Implementation (NSDI '06)*. San Jose, CA, USA: USENIX Association, 2006. [Online]. Available: https://www.usenix.org/conference/nsdi-06/subtleties-tolerating-correlated-failures-wide-area-storage-systems

[36] D. Leong, A. G. Dimakis, and T. Ho, "Distributed storage allocations," *IEEE Transactions on Information Theory*, vol. 58, no. 7, pp. 4733–4752, 2012.

[37] T. Batu, S. Kannan, S. Khanna, and A. McGregor, "Reconstructing strings from random traces," in *Proceedings of the 15th Annual ACM-SIAM Symposium on Discrete Algorithms (SODA)*, New Orleans, LA, USA, 2004, pp. 910–918.

[38] S. R. Srinivasavaradhan, S. Gopi, H. D. Pfister, and S. Yekhanin, "Trellis BMA: Coded trace reconstruction on IDS channels for DNA storage," in *Proc. IEEE International Symposium on Information Theory (ISIT)*. IEEE, 2021, pp. 2453–2458.

[39] O. Sabary, A. Yucovich, G. Shapira, and E. Yaakobi, "Reconstruction algorithms for DNA-storage systems," *Scientific Reports*, vol. 14, no. 1, p. 1951, 2024.

[40] C. Rashtchian, K. Makarychev, M. Racz, S. Ang, D. Jevdjic, S. Yekhanin, L. Ceze, and K. Strauss, "Clustering billions of reads for DNA data storage," in *Advances in Neural Information Processing Systems 30 (NIPS 2017)*, 2017. [Online]. Available: https://papers.nips.cc/paper_files/paper/2017/hash/ab7314887865c4265e896c6e209d1cd6-Abstract.html

[41] D. Bar-Lev, I. Orr, O. Sabary, T. Etzion, and E. Yaakobi, "Scalable and robust DNA-based storage via coding theory and deep learning," *Nature Machine Intelligence*, vol. 7, no. 4, pp. 639–649, 2025.

[42] C. Delahaye and J. Nicolas, "Sequencing DNA with nanopores: Troubles and biases," *PLOS ONE*, vol. 16, no. 10, p. e0257521, 2021.
[43] R. R. Wick, L. M. Judd, and K. E. Holt, "Performance of neural network basecalling tools for Oxford Nanopore sequencing," *Genome Biology*, vol. 20, no. 1, p. 129, 2019.
[44] S. Bubeck, D. Ernst, and A. Garivier, "Optimal discovery with probabilistic expert advice: Finite time analysis and macroscopic optimality," *Journal of Machine Learning Research*, vol. 14, no. 17, pp. 601–623, 2013. [Online]. Available: https://www.jmlr.org/papers/v14/bubeck13a.html
[45] FML, ETH Zurich, "DT4DDS – characterization of errors and biases in DNA data storage," European Nucleotide Archive, study PRJEB65931, 2023, sequencing data of Gimpel et al., Nat. Commun. 14:6026 (2023); first public 2023-09-08. [Online]. Available: https://www.ebi.ac.uk/ena/browser/view/PRJEB65931
[46] I. S. Reed and G. Solomon, "Polynomial codes over certain finite fields," *Journal of the Society for Industrial and Applied Mathematics*, vol. 8, no. 2, pp. 300–304, 1960.
[47] B. Efron and R. J. Tibshirani, *An Introduction to the Bootstrap*. Springer US, 1993.
[48] C. J. Clopper and E. S. Pearson, "The use of confidence or fiducial limits illustrated in the case of the binomial," *Biometrika*, vol. 26, no. 4, pp. 404–413, 1934.
[49] T. Gneiting and A. E. Raftery, "Strictly proper scoring rules, prediction, and estimation," *Journal of the American Statistical Association*, vol. 102, no. 477, pp. 359–378, 2007.
[50] Technion – Israel Institute of Technology, "Binned Nanopore/Illumina reads from DNAformer, split by the data type," Zenodo dataset, 2025. [Online]. Available: https://doi.org/10.5281/zenodo.17473983